

UNIVERSIDAD EUROPEA DE MADRID

FACULTY OF SOCIAL SCIENCES

POSTGRADUATE PROGRAMME IN DATA ANALYTICS

MASTER'S DEGREE DISSERTATION

SENTIMENT-DRIVEN INVESTMENT SIGNAL: A PRESCRIPTIVE MACHINE LEARNING APPROACH INTEGRATING FINBERT, TBL, VIX, AND RSI FOR ALPHA GENERATION IN THE BIG TECH CLUSTER (2018–2023)

María Gracia Morales Cabrera

Sandro Miguel Da Costa Vaz Dias

Supervised By:

Alvaro Orgaz Exposito

ACADEMIC YEAR 2025 - 2026

TITLE: SENTIMENT-DRIVEN INVESTMENT SIGNAL: A PRESCRIPTIVE MACHINE LEARNING APPROACH INTEGRATING FINBERT, TBL, VIX, AND RSI FOR ALPHA GENERATION IN THE BIG TECH CLUSTER (2018–2023)

AUTHOR: MARÍA GRACIA MORALES CABRERA AND SANDRO MIGUEL DA COSTA VAZ DIAS.

POSTDEGREE: PRIVATE MASTERS DEGREE IN DATA ANALYTICS.

PROJECT SUPERVISORS(S) ALVARO ORGAZ EXPOSITO.

DATE: JULY 2026

ABSTRACT

The following master thesis recommends a prescriptive investment framework that joins financial news sentiment analysis and quantitative market indicators, with the objective of generating understandable investment signals. This study tackles the limitations of conventional predictive models, that mostly rely on structural numerical information or data and ignores the value contained in unstructured financial news frequently. The methodology of this study combines Natural Language Processing (NLP) procedures, technical market indicators, and supervised machine learning in order to sustain investment decisions based on data.

To develop this study a five-year dataset (2018-2023) was used containing financial news and historical market data for a portfolio of greatly covered technology stocks. Using the FinBert language model, along with integrated with market features, including the Volatility Index (VIX), the Relative Strength Index (RSI), Average Daily, Trading Volume (ADTV), and volatility, financial sentiment was extracted and then, a Triple Barrier Labeling strategy was applied in order to generate realistic targets adjusted to risk. An XGBoost classifier was then constructed to produce investment signals. A “portfolio backtesting” was applied in order to evaluate the strategy in which the strategy of this study was put up against the market benchmark.

With the results of this study it was possible to prove the viability of integrating textual sentiment with quantitative indicators with sight of a cohesive predictive framework.

The study paper shows the potential of combining artificial intelligence with sentiment analysis to boost predictive financial modeling, while fusing practical risk-management principles.

Keywords: Machine Learning, XGBoost, Algorithmic Trading, Natural Language Processing, Quantitative Finance, Financial Sentiment Analysis, Financial Markets.

ACKNOWLEDGEMENTS

Before all, we would like to extend our gratitude to God for his guidance, and blessings all along our journey.

Furthermore, we would equally like to share sincere words of appreciation to our thesis coordinator and supervisor, Professor Álvaro Orgaz Exposito, for his guidance, and support, throughout our project.

Finally, we would like to express our gratitude to the college and our professors during our Master's programme for endowing us with a fulfilling academic environment and for providing us with the adequate knowledge and skills that made this study possible.

"*The important thing is not to stop questioning. Curiosity has its own reason for existing.*"

— Albert Einstein

SUMMARY TABLE

	DATA
Full name:	María Gracia Morales Cabrera and Sandro Miguel Da Costa Vaz Dias
Project title:	Sentiment-Driven Investment Signal: A Prescriptive Machine Learning Approach Integrating FinBERT, TBL, VIX, and RSI For Alpha Generation In The Big Tech Cluster (2018–2023)
Project supervisors:	Alvaro Orgaz Exposito
The project has been carried out in collaboration with, or commissioned by, a company:	NO
The project has implemented a product:	NO
The project has involved the development of research or innovation:	YES
General objective of the project:	Construct a prescriptive analytics model that joins sentiment analysis based on Natural Language Processing with macroeconomic and technical indicators. The aim is to improve investment decision-making and protect capital in highly volatile financial markets, particularly in non-stationary environments.

Index

ABSTRACT 3

Chapter 1. PROJECT OVERVIEW 11

1.1 Context and justification 11

1.2 Problem statement 11

1.3 Project objectives 11

1.4 Results obtained 12

1.5 Structure of the report 12

Chapter 2. BACKGROUND / STATE OF THE ART 13

2.1 State of the art 13

2.2 Context and justification 17

2.3 Problem statement 18

Chapter 3. OBJECTIVES 21

3.1 General Objectives 21

3.2 Specific objectives 21

3.3 Project Benefits 21

Chapter 4 PROJECT DEVELOPMENT 23

4.1 Project planning 23

4.2 Description of the Solution, Methodologies and Tools Employed 27

4.2.1 Operational Workflow 27

4.2.2 The financial dataset 28

4.2.3 Data Engineering and out-of-core processing 28

4.2.4 Deep Learning and Sentiment Extraction (FinBert) 29

4.2.5 Features and variable engineering 30

4.2.6 Triple Barrier Labeling (TBL) Method as a dynamic target labeling framework 33

4.2.7 The XGBoost Algorithm 35

4.2.8 The Kelly Criterion for portfolio capital allocation and position size per trade 36

4.2.9 Portfolio Backtesting Engine....37

4.2.10 Practical example and investor operational rules....40

4.3 Required Resources....42

4.4 Budget....43

4.5 Feasibility....44

4.6 Project results....45

4.6.1 Alternative data validation and sentiment distribution....45

4.6.2 Predictive Engine Development and Signal Translation....50

4.6.3 Institutional Performance and Stress Testing....58

Chapter 5. DISCUSSION....73

5.1 Discussion of the results....73

5.2 Study Limitations and Practical Constraints....77

Chapter 6. CONCLUSIONS....81

6.1 Conclusions of the work....81

6.2 Personal conclusions....82

Chapter 7. FUTURE WORK....83

Chapter 8. REFERENCES....84

Chapter 9. APPENDICES....93

List of Figures

Figure 1. Alternative Data Adoption Rate by Financial Analysts (2010 - 2026)..... 18

Figure 2. Lagged Sentiment Distribution (T-1)..... 46

Figure 3. Sentiment Score Distribution using FinBERT..... 47

Figure 4. News Polarity Composition per Asset (Percentage labels)..... 48

Figure 5. Mega - Cap Tech Sentiment Momentum..... 49

Figure 6. Algorithmic Feature Hierarchy: What Drives the Model?..... 52

Figure 7. XGBoost Feature Correlation Matrix..... 54

Figure 8. Explainable Alpha: NLP Sentiment to Profit Conversion Pipeline..... 56

Figure 9. Defensive Alpha: NLP Sentiment to Capital Preservation Pipeline..... 57

Figure 10. Strategy Operational Performance & Expectancy Dashboard..... 61

Figure 11. Cumulative Wealth Evolution: XGBoost Predictive Framework vs S&P 500..... 63

Figure 12. Systematic Portfolio Performance Attribution Matrix..... 64

Figure 13. Comparative Drawdown Analysis: Trained Model vs S&P 500..... 65

Figure 14. Alpha Distribution: Net Profit Attribution by Asset Ticker (USD)..... 66

Figure 15. Stress Testing Matrix: Dual-Axis Sensitivity Heatmap of the XGBoost Strategy's Model Risk vs. Capital Sizing..... 68

Figure 16. Execution Robustness: Portfolio Survivability Under Severe Liquidity Crises..... 70

Figure 17. Parametric Sensitivity Analysis of the Time Barrier Horizon chart..... 71

List of Tables

Table 1. Budget..... 43

Table 2. Classification Report (Test set)..... 51

Table 3. Risk/Performance Metric of the Model vs S&P 500 Benchmark..... 59

Table 4. Systematic Risk and Benchmark Correlation (CAPM)..... 60

Table 5. Operational and Execution Metrics..... 62

Table 6. Alpha Distribution: Net Profit Attribution by Asset Ticker (USD)..... 66

Table 7. Sensitivity Analysis Table, risk target multiplier and max capital cap allocation per trade 68

Table 8. Sensitivity Analysis Table, risk target multiplier and max capital cap allocation per trade 70

Table 9. Parametric Sensitivity Analysis of the Time Barrier Horizon table..... 71

Chapter 1. PROJECT OVERVIEW

1.1 Context and justification

As a result of the vast amounts of digital financial information, processing textual data processing, such as financial news, has gained more importance. Traditional financial models mostly base on numerical variables and have a habit to omit the informational utility of text. The growth of Natural Language Processing (NLP) and machine learning techniques have supported the extraction of valuable news from large-scale textual data, allowing new possibilities for integrating unstructured data into financial analysis (Loughran & McDonald, 2011; Liu et al., 2019).

TSuch an approach is essential due to the need for more understandable data-driven methodologies that can combine textual and numerical data when creating a predictive financial framework.

1.2 Problem statement

Financial markets have as one of its main characteristics the sensibility to news and information, yet the majority of quantitative models do not totally consider textual data.

Although NLP techniques can be used to facilitate the extraction of sentiment from text, incorporating these signals into predictive models is still a challenge as a result of issues such as data processing, temporal alignment and the leakage of data.

This study focuses on the issue of financial news conversion into structured sentiment signals along with macroeconomic and technical indicators for the use of financial analysis.

1.3 Project objectives

The main objective of this project is exploration of the integration of news-based sentiment along with macroeconomic and technical indicators in a quantitative financial framework by using machine learning methods.

This study focuses on the processing of financial news, generation of sentiment-based characteristics, and including these into a predictive modelling environment, while ensuring the maintenance of methodological consistency.

1.4 Results obtained

This study successfully produced and validated a prescriptive investment framework that not only integrates financial news sentiment with quantitative market indicators but also generates understandable investment signals. The project methodology unites sentiment analysis based on FinBERT, technical indicators, Triple Barrier Labeling, and an XGBoost classifier. This combination can demonstrate the potential of transforming unstructured and confusing financial data into structured information supported by information. The process of evaluation incorporated portfolio backtesting and a comparison against the S&P 500 benchmark in order to demonstrate the framework's ability to mediate effective predictive financial analysis. This is achieved by the integration of realistic risk management methodologies all while maintaining the interpretability of the model.

1.5 Structure of the report

This report is divided into seven chapters. The following chapter covers the theoretical foundations and related work on financial sentiment analysis, machine learning, and predictive investment models. Chapter 4 gives an understandable general overview of the project's planning and development methodology, including the resources, tools and implementation process. Chapter 4 is concerned with the design, implementation and evaluation of the proposed framework. This chapter reviews the processing of data, the extraction of sentiment, predictive modelling, and the analysis of the project's performance. Chapter 6 provides the main conclusions while discussing the limitations related to the study. Chapter 7 then shares recommendations for future projects and research.

Chapter 2. BACKGROUND / STATE OF THE ART

2.1 State of the art

Due to the rapid growth of unstructured financial data there has been an important paradigm shift in algorithmic trading, this has exposed the vital limitations of quantitative models and driven the financial industry toward advanced natural language processing architectures. These architectures have the capability to assimilate path-dependent predictive modelings rapidly. Globally the financial landscape is currently under a state of constant transformation and mutation, which is being driven by the revolution of 'Big Data' and the emergence of high-frequency information ecosystems. In order to be able to grasp the actual state of knowledge, it is of high relevance to trace the evolution of financial sentiment analysis in recent years, from primitive, lexicon-based heuristics, through domain-adapted discriminative transformers, to generative large language models (LLMs). This progression can be shown by a prevailing 'modality gap' in the processing of information and a structural deficiency in predictive logic: notably, the dependence on frameworks fixed on horizon rather than realities that are path dependent. A price target is predicted to be reached at a specific future point in time, while disregarding the path-dependency of prices. In sectors with high volatility, which is the case of those involving major technology companies, the inability to consider intraday variability of prices resulting in stop-loss orders can lead to high performance expectations that are not in line with actual outcomes (Bhuvaneshwari, 2025; Goldstein et al., 2021).

The ecosystem in modern days generates more information every two days than people managed from its inception up to 2003, which permanently breaches historical limits of global data generation. This is not due to merely a rapid acceleration in volume; it also indicates a major structural shift away from past econometric norms. Goldstein, Spatt and Ye (2021) identify three key features of the big data paradigm: large size, high dimension and complex structure.

The large size attribute refers to the availability of massive data volumes that effectively eliminate the selection biases inherent in smaller data that allows predictive algorithms to train on population-level data representations. Secondly, the high dimension attribute describes statistical environments where the number of potential variables exponentially exceeds the number of traditional observations that challenges classical financial models because they

often suffer from overfitting when exposed to hyper-dimensional feature spaces. Finally, complex structure deals with the raw, disorganized reality of modern finance: unprocessed audio from earnings calls, dense regulatory filings, and an endless textual flood of analyst reports and social media discourse. Mapping the true state of the market now demands a dual approach. It is important that modern algorithms do not rely on price history alone, as they must integrate quantitative price history with the qualitative sentiment extracted from complex, unstructured sources to make an accurate analysis (Goldstein et al., 2021).

In this context, market efficiency is no longer defined only by the speed of trade execution, but by the capacity to rapidly assimilate information. As Araci (2019) notes, prices in an open market reflect all available information regarding assets exchanged in an economy. When new information becomes available, actors update their positions, and prices adjust accordingly. However, the definition of "new information" is evolving as retrieval technologies advance; this is why the early adoption of these technologies may provide a critical advantage in the short term. With an unprecedented amount of financial text, including news, analyst reports, and official announcements, if this is being created every day, the task of manually deriving actionable insights is too immense for any single entity.

This challenge is intensified by a modality gap between qualitative financial information and quantitative trade execution. Conventional sentiment analysis models are general-purpose linguistic frameworks that are inadequate for the interpretation of complex financial terminology. For instance, financial terms such as "volatility" or "exposure" necessitate contextual information to ensure accurate interpretation. Consequently, conventional Natural Language Processing (NLP) models, which represent a multidisciplinary field encompassing linguistics, computer science and artificial intelligence, are unable to adequately address such nuances due to their lack of domain-specific intelligence, in consequence, these models often exhibit inaccuracies, compromising their reliability (Omoseebi et al., 2023).

FinBERT is a language model specific to a domain conceived to shorten the knowledge gap in the field of finance. It is a pre-trained deep-layer neural network built with 4.9 billion tokens of financial text data from the following sources: SEC filings, analyst reports, and earnings call transcripts (Yang et al., 2020). In the most recent advancement of knowledge, Generative Large Language Models (LLMs) have surpassed the previous state of knowledge by providing better reasoning ability, but the low inference latency and computational efficiency of FinBERT make

it especially useful in very high-frequency trading environments (Muhammad et al., 2025). To address the issue of models inadvertently trading on non-actionable boilerplate text, models like FinBERT are applied with filtering techniques that can separate the relevant financial signals from the vast amount of noise in the market.

Several recent studies have found many challenges in financial machine learning and sentiment analysis. Araci's research titled “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models” introduced FinBERT, establishing that pre-trained language models fine-tuned on domain-specific corpora (in this case, specialized in financial task) significantly outperform traditional machine learning methods in financial sentiment analysis. Araci conducted experiments to investigate several aspects of the model, including the effects of further pre-training on a financial corpus, training strategies to prevent catastrophic forgetting, and fine-tuning only a small subset of model layers to decrease training time and avoid a significant drop in performance. Araci utilized two primary datasets: first, “TRC2-financial”, a filtered subset of 46,143 documents derived from a larger Reuters corpus of 1.8 million news articles published between 2008 and 2010, and second, the “Financial PhraseBank”, which contains 4,845 English sentences selected randomly from financial news found on the LexisNexis database. Araci demonstrated that FinBERT achieved an impressive 97% accuracy and an F1 score of 0.95 on the Financial PhraseBank dataset (for sentences with 100% annotator agreement), improving upon previous state-of-the-art machine learning benchmarks by 14 percentage points, while also outperforming other transfer learning baselines like ELMo and ULMFit[1] (Araci, 2019).

Recent studies have shifted from predictive to prescriptive modeling. Cezanne made a study (2024) titled “Large language model-driven sentiment analysis for exploring the influence of social media and financial news on stock market development” transformed prediction into action as this project evaluated how sentiment driven by an LLM such as FinBERT has directly affected stock prices, highlighting how this model FinBERT is superior as its Mean Squared Error (MSE) is lower, which means that it has a lower margin of error than conventional methods, and the R-squared statistic linking news articles with stock price movement is more accurate, so FinBERT has a more reliable correlation in explaining why a stock price moved.

[1]Both ULMFiT and ELMo represent a crucial turning point. By introducing transfer learning and bidirectional context analysis, they built the technical foundation for the later development of transformer-based models, such as BERT and FinBERT. Currently, these advanced models dominate prescriptive analytics and signal generation in financial markets (Singh, 2024).

A similar study, titled 'Market Reactions to Earnings Calls: Evidence from GPT-4o-derived sentiment analysis', developed by Van Mook (2025), gave more strength to the usefulness of advanced semantic extraction in the field of finance. This study analysed corporate financial earnings calls, which are formal meetings where company executives discuss financial performance with analysts. The author showed that generative AI models such as GPT-4o are far superior to traditional methods based on dictionaries, which only took into consideration positive or negative words without fully understanding the actual context of the conversation. Furthermore, the study revealed a strong correlation between detected sentiment and short-term abnormal returns. In other words, the tone identified by the model can accurately predict unusual spikes or drops in stock prices immediately after earnings announcements. This proves the reliability of modern Large Language Models in translating complex corporate speech into short-term market signals.

In the same context, the research paper entitled 'Financial Sentiment Analysis: Evaluated on the Stock Market " studied the impact of financial sentiment prediction on the stock market. A comparative study of large language models and transformer-based sentiment classifications', by Alvim de Faria and Lundquist (2025), studied the performance of LLMs such as next-day predictors of stock market performance based on the analysis of multiple inputs, like news articles and social media, in comparison to transformer-based models such as FinBERT and DistilRoBERTa. The results concluded that the best predictive performance, with an accuracy rate higher than 90%, was achieved by combining both AI-generated sentiment features and machine learning classifiers. The study showed that FinBERT was very effective when applied to structured financial documents while generative models were more successful at generalizing from informal sources such as social media to predict stock prices.

Another related research that is titled as "Financial sentiment analysis in annual reports of BIST100 companies: Comparison of dictionary-based and deep learning-based methods " and developed by Tür (2024), compared dictionary (Attribute Value Based) and deep learning based approaches to analyze corporate annual reports. He determined that deep learning outperformed dictionary-based approaches at capturing the shades of complex financial language. These findings not only validated the effectiveness of deep learning but also revealed a strong correlation between the sentiment trends obtained from financial documents and economic indicators like GDP (Gross Domestic Product; an indicator representing the total

monetary value of all goods and services produced within a country's borders). These academic works validate the ongoing industry transition from generic NLP tools to highly domain-adapted forecasting systems.

2.2 Context and justification

Financial markets have mutated into data-driven environments, as many investment decisions may be affected by a great quantity of numerical indicators and information from financial news and digital media. Traditional quantitative models typically have been centred on numerical variables such as prices, returns and volatility. However, recent research has indicated that textual data and information as well as market sentiment can also offer valuable insights into investor expectations and market behaviour (Loughran & McDonald, 2011).

It is of the same importance to bring up that the use of natural language processing (NLP) and deep learning techniques has majorly improved sentiment analysis in finance. In this context, FinBERT (a transformer-based model) can bring out sentiment and contextual meaning from financial texts (Liu et al., 2019). Machine learning models such as XGBoost nowadays excel at managing large, complex financial datasets featuring relationships that are non linear and high dimensionality (Chen & Guestrin, 2016).

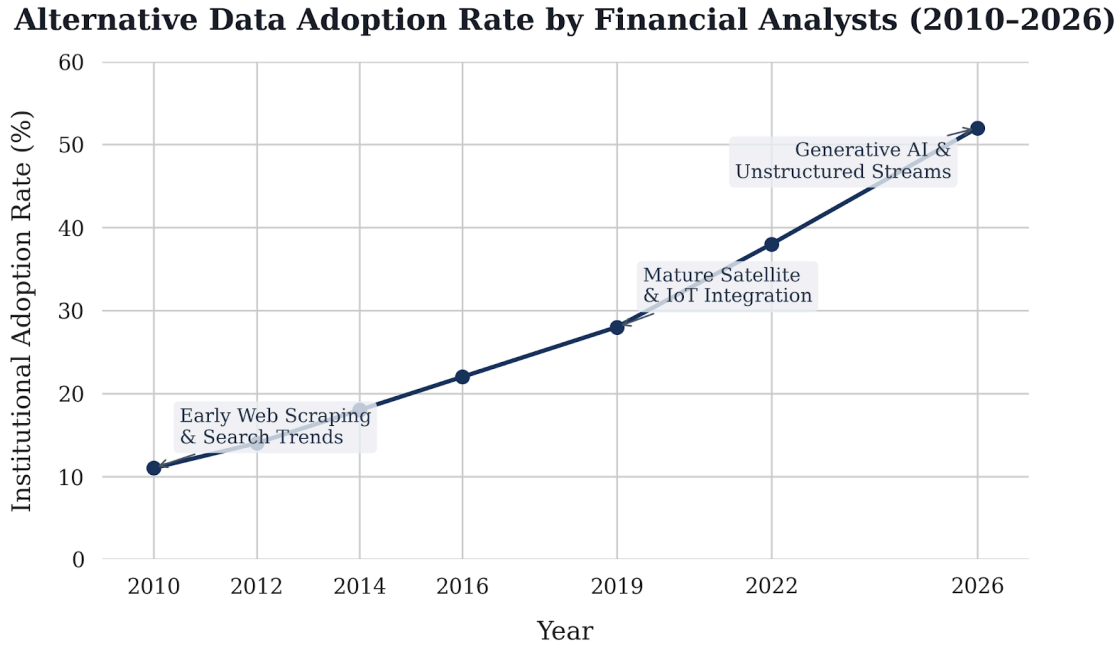
Recent studies have explored more and more the integration of textual sentiment into predictive financial frameworks. However, these studies still encounter significant challenges in processing and handling large-scale news datasets, aligning textual information with market data and preventing data leakage in predictive modelling.

This study tackles these challenges by taking a deep dive and analysing a large-scale financial news dataset comprising millions of records relating to major technology companies such as Apple, Microsoft, Nvidia, Tesla, Amazon, Google and Intel. The project has the goal of exploring the feasibility of integrating unstructured textual information into quantitative financial analysis and combining sentiment indicators extracted from financial news with historical market data.

The dependency for analytical methods that can handle and process vast amounts of data all the while remaining methodologically rigorous and scalable is the basis of this research. Additionally, this research relates to the next step in the evolution of artificial intelligence in

finance, focusing on the application of structured and unstructured data in financial prediction models.

Figure 1. Alternative Data Adoption Rate by Financial Analysts (2010 - 2026)



Note: This figure indicates the institutional adoption curve of alternative data by financial analysts from 2010 to 2026. The trajectory of the curve shows the shift from basic web-scraping techniques to advanced, AI-driven processing of unstructured data. Adapted from data by Chi, Hwang and Zheng (2024).

2.3 Problem statement

Even though unstructured big data and sentiment analysis have an important impact on stock prices, there is still a discrepancy. This discrepancy lies in accurately translating financial news and narratives into solid profitable and executions while maintaining low risk. For this it is necessary to apply existing algorithmic trading capabilities to understand financial rhetoric within specific domains, all the while also considering the chaotic and path-dependent nature of stock price volatility in the real world financial markets.

The existing literature reveals distinct deficiencies that might limit models:

First, the majority of automated trading systems continue to rely on traditional, wide-ranging machine learning classifiers or lexicon-based (dictionary) techniques to read financial news or

social media posts. As a result, many of these standard models do not possess the highly specialized linguistic capabilities needed to interpret the dense, contradictory jargon that is prevalent in the financial markets. Due to this lack of specialized linguistic capabilities, many of these conventional models are faced with a serious modality gap, leading to a lack of distinction of polarity in terms that would otherwise convey considerable differences in financial polarity depending on the context. Consequently, this lack of semantic disambiguation will produce highly noisy (inaccurate) sentiment signals. Furthermore, there is a significant deficit of solutions that can provide seamlessly deployed domain-adapted, bidirectional deep learning models that accurately process these high volume informational noise, in consequence, creating high-frequency data sets (Dong et al., 2024).

Despite extracting sentiment accurately, utilizing it to forecast asset prices exposes a massive structural flaw in legacy models: the reliance on fixed-horizon logic. The absolute frontier of quantitative machine learning has therefore adopted path-dependent labeling methodologies such as Triple Barrier Labeling (TBL) (Fu et al., 2024) to overcome the illusion of profitability resulting from ignoring intraday drawdowns. TBL models the exact mechanical constraints of a live trading environment by setting dynamic horizontal barriers for profit-taking and stop-losses, as well as a vertical time barrier, evaluating the actual price trajectory rather than an arbitrary future point (Kang, 2025). The integration of FinBERT's domain-specific sentiment extraction and Triple Barrier Labeling's path-dependent framework is best validated in highly volatile environments, such as the global big tech industry. Due to intraday price swings driven mostly by qualitative news events in regards to supply chains, geopolitical tensions, technological innovation and other factors, successfully predicting momentum in the tech sector demands a precise processing of advanced semantic parsing and path-dependent risk mechanics (Bhuvaneshwari, 2025).

It is of great importance to note that depending solely on news sentiment is insufficient as it does not consider broader market conditions and technical extremes that could impact price movements. As precaution and in order to safeguard capital and manage risk, it is essential to unify sentiment analysis within macroeconomic realities and technical indicators. The indicators are the Volatility Index, also referred as the market´s "fear gauge", which measures the 30-day expected volatility derived from S&P 500 options to identify periods of systemic risk and risk aversion, also, technical indicators such as the Relative Strength Index (RSI), which is a

a critical feature for capturing short-term mean-reverting tendencies within the asset's price dynamics and identify overbought or oversold conditions (Wilder, 1978), and 20-day Average Daily Trading Volume (ADTV) that indicates the liquidity profile of an asset. High ADTV facilitates efficient order execution and minimizes price impact (slippage), whereas low ADTV introduces liquidity friction, significantly complicating the entry and exit of positions (Harris, 2003). Both indicators are required to distinguish meaningful trend participation from low-liquidity noise and ensure that signals are supported by significant market interest and liquidity.

Therefore, the guiding research question this work seeks to answer is:

How does the integration of Sentiment Analysis (driven by Natural Language Processing and extracted from financial news) with technical and macroeconomic indicators (specifically the VIX, RSI, and TBL) enhance the robustness of a prescriptive analytics model for investment decision-making and capital preservation in non-stationary markets?

Chapter 3. OBJECTIVES

3.1 General Objectives

This project's main goal is to construct a prescriptive analytics model that joins sentiment analysis based on Natural Language Processing with macroeconomic and technical indicators. The aim is to improve investment decision-making and protect capital in highly volatile financial markets, particularly in non-stationary environments.

3.2 Specific objectives

- To quantify unstructured market sentiment by applying FinBERT to a historical financial news dataset, extracting high-resolution polarity signals across a cluster.
- To develop and train a machine learning-based classification engine (XGBoost) that can translate sentiment and technical feature matrix into prescriptive investment signals.
- To evaluate the structural robustness, predictive accuracy, and the capability to preserve capital through multi-dimensional parameter sensitivity and liquidity stress testing, benchmarking its risk-adjusted performance directly against the S&P 500.

3.3 Project Benefits

This initiative suggests a framework that illustrates the availability of a number of methodological, practical as well as academic advantages and proves that the combination of financial news sentiment with quantitative market indicators is an effective avenue for the integration of both unstructured textual output as well as structured financial figures into a uniform predictive system. The proposed multimodal technique surpasses conventional systems reliant only on historical prices.

From a methodological perspective, the project establishes a complete and reproducible pipeline that encompasses large-scale data preprocessing, financial sentiment extraction using FinBERT, feature engineering, volatility-adjusted target generation through the Triple Barrier Labeling methodology, and predictive modeling using XGBoost. Incorporating machine learning techniques that can be explained ameliorates the model's transparency by recognizing the variables that most significantly betters investment decisions.

From a pragmatic point of view, the framework translates raw financial news into comprehensible investment signals and amalgamates realistic risk management mechanisms. Integrating market indicators such as the Volatility Index (VIX), the Relative Strength Index (RSI), Average Daily Trading Volume (ADTV) and volatility measures allows the model to assess market conditions from multiple perspectives. Furthermore, implementing portfolio backtesting, transaction costs, slippage and fractional Kelly position sizing improves the realism of the proposed investment strategy, facilitating its evaluation under conditions that closely mimics those of real financial markets.

In conclusion, this study contributes significantly to the emerging research field of machine learning applications in finance. The merger between natural language processing and machine learning is demonstrated through the implementation of prescriptive financial decision-making support systems. Additionally, the modular architecture used throughout the study provides a flexible base for future research in the area of machine learning applications in finance that can be scaled with the integration of new assets and new sources of data or some other improvements.

Chapter 4 PROJECT DEVELOPMENT

4.1 Project planning

This project was developed using an organized, iterative workflow that joins academic research, data processing in large scale, financial analysis, and machine learning implementation. The project was organized into various phases, each related with specific technical activities aligned towards the research necessary in order to achieve the tentative objectives.

4.1.1 Literature Review and State of the Art

During the initial phase of the project, extensive research was carried out on academic literature and data in the field of financial sentiment analysis, natural language processing (NLP), as well as the use of machine learning in finance application. More specifically areas that were looked into include research on the transformer-based language models as well as predictive financial modeling and trading models that are based on sentiment analysis. In addition to this, literature was checked for methodologies such as Triple Barrier Labelling (TBL), regular trends in the financial markets and methods of risk-adjusted portfolio analysis.

A review of the literature, technical papers, and benchmark solutions on FinBERT, XGBoost, and financial data engineering content was also done on this stage to ensure a strong theoretical and methodological basis for the project.

4.1.2 Data Acquisition and Preliminary Technical Analysis

The next phase focused on obtaining, analysing and validating the datasets that were used throughout this project, which included large amounts of financial news datasets covering roughly five years of historical records, as well as market data extracted via external financial interfaces and APIs.

A preparatory technical analysis was performed in order to validate the integrity of the datasets. Due to the vast amounts of news records, additional effort was necessary in order to assess hardware limitations and define scalable processing strategies. A high level of effort was necessary in this due to the complexity of managing diverse data sources.

4.1.3 Exploratory Data Analysis (EDA)

Following the obtention and extraction of the data, a thorough exploratory data analysis (EDA) was conducted to understand the characteristics, specifications, and distribution of the information and data collected. This analysis involved evaluating publication frequency, stock coverage, the temporal distribution of news articles, missing values and sentiment tendencies across different companies.

As a result of memory limitations because of the massive size of the dataset, it was necessary to employ a processing strategy to analyse the millions of news records in an efficient manner without exceeding the hardware's capabilities.

4.1.4 Data Engineering and Feature Construction

The fourth stage focused on developing the data engineering pipeline and constructing the final analytical dataset. This stage had as objective the preprocessing of textual data, filtering relevant records, normalising timestamps and aggregating sentiment information at a daily level.

At the same time, quantitative financial indicators were added into the framework and they were also temporally aligned:

- Volatility Index (VIX)
- Relative Strength Index (RSI)
- Average Daily Trading Volume (ADTV)
- Rolling volatility measures,
- Lagged returns and temporal features

It is important to consider that ensuring correct temporal alignment between financial news and market data was crucial to prevent data leakage. It was also necessary to validate the chronological consistency and accuracy of the data and build a reliable base before modelling.

4.1.5 Sentiment Analysis Implementation

This fifth phase focused on establishing, implementing, and integrating the FinBERT transformer-based language model in order to extract financial sentiment. An understandable database of financial news articles was analysed thoroughly in detail to extract sentiment

classifications and probability scores, which in turn were then used to establish whether positive, negative or neutral sentiment was present.

This extracted sentiment data was then aggregated and transformed into structured numerical indicators. To ensure semantic consistency and the appropriate representation of sentiment signals across the selected portfolio of technology companies, additional validation procedures were conducted.

Implementing this required significant computational, and methodological effort due to the large size of the textual dataset, and the complexity of processing domain-specific financial language.

4.1.6 Labeling Strategy and Predictive Framework Design

After constructing the feature set, the project's next steps went on to designing realistic, and accurate predictive targets by implementing the Triple Barrier Labelling (TBL) methodology. Unlike other conventional binary labelling methodologies, TBL introduced volatility-adjusted stop-loss and profit-taking boundaries, ensuring the simulation of practical trading constraints.

At this stage , the parameters and frameworks of the labelling had to be calibrated, and the volatility procedures estimated. Additionally, the target distributions were validated. This phase demanded a medium to high level of effort, in order to balance the goal of predictive realism with maintaining dataset stability.

4.1.7 Machine Learning Model Development

The seventh phase involved developing the predictive engine using the XGBoost classification algorithm. Large amounts of time and effort were dedicated to training, validation, and procedures were then executed to evaluate the model's robustness, the importance of the features, and the consistency of the predictions.

The implementation concerned:

- Hyperparameter tuning
- Cross-validation procedures
- Training/testing data partitioning

- Performance monitoring
- Feature contribution analysis

A particular emphasis was placed on the integration of textual sentiment variables with conventional quantitative indicators for the purpose of evaluating the incremental value provided by information derived from news sources.

4.1.8 Validation, Benchmarking, and Performance Evaluation

The eight and final analytical phase focused on testing and evaluating the predictive and financial performance of the project's proposed framework. The model's outputs were then compared with the S&P 500 benchmark using a range of predictive and financial metrics adjusted to risk. These metrics included:

- Sharpe Ratio
- Sortino Ratio
- Classification accuracy
- Return stability
- Signal consistency

In order to assess the robustness of the system under different market conditions and to evaluate the practical applicability of the generated investment signals, additional validation procedures were conducted.

4.1.9 Phase 9 – Documentation and Thesis Preparation

The ninth and final stage of the research process involved compiling and aggregating results, interpreting findings, preparing data visualisations, and drafting the thesis document. This part involved organising methodological explanations, discussing limitations, validating references, and preparing presentation materials.

Throughout the whole project, in order to ensure methodological coherence and alignment between the technical implementation and the research objectives continuous meetings and reviews were conducted.

4.2 Description of the Solution, Methodologies and Tools Employed

To address the limitations, and challenges of traditional trading models, a prescriptive trading model has been developed and integrated with deep learning, sentiment analysis, and indicators of macroeconomics, volatility and technical analysis. This step allows the framework to generate trading signals with high conviction and incorporates the triple barrier method as a risk manager and sets a take-profit and stop-loss limits. The entire technical pipeline of this project is executed within a Google Colab environment with Graphics Processing Unit (GPU) acceleration (T4), as this shifts critical deep learning and sentiment analysis processes, such as matrix operations and linear algebra, which are heavy computational workloads. The T4 ensures the time required for training complex models is reduced, and processes large datasets, which is crucial to the project's development.Google Drive is also used for saving, maintaining data lineage, and intermediate asset serialisation.

4.2.1 Operational Workflow

In order to ensure the integrity of data and prevent any kind of bias, the model follows five processing layers:

1. **Data ingestion based on Extract, Transform, and Load (ETL):** It is necessary to filter the massive dataset to identify high-density stock news, extract them, and couple them with the Yahoo Finance API in Google Colab for daily Open, High, Low, Close, and Volume (OHLCV) information from financial markets and macroeconomic data.
2. **Natural Language Processing (NLP):** The key in this step is to make deep learning text classification using the FinBERT Transformer model to classify and assign sentiment scores to daily news.
3. **Temporal Stock Price Synchronization (Lead - Lag):** In order to predict sentiment analysis, stock movement, and price, a chronological alignment is needed. For this, it is important to use a backward shifting merge function to map and identify news sentiment at Day T against the market opening price at Day T + 1 (Qinyun Zen, 2019).
4. **Triple Barrier Method (TBL) to manage risk:** TBL is risk-adjusted labeling that is dependent on the path of the trade, which classifies and establishes volatility-adjusted profit taking and stop loss boundaries.

5. **Prescription layer (XGBoost):** In this layer, the ensemble tree-based classification and confidence-related decision rules are used to output actionable signals (buy, sell, or hold) under a simulated but realistic trading environment.

4.2.2 The financial dataset

The primary dataset that provided the qualitative corpus is the Financial News and Stock Price Integration Dataset (FNSPID) is a large scale repository created by Zihan Dong, Xinyu Fan, and Zhiyuan Peng (2024) that contains 15.7 million financial records that are aligned in time with 29.7 million daily price records from 1999 to 2023 across 4,775 companies that have belonged to the S&P500. These headlines are extracted from premier financial web portals like Nasdaq, Bloomberg, Reuters, and Benzinga. This is an excellent dataset because it includes source identifiers, URLs, exact timing, and tickers to map stocks, also, a key benefit of the dataset is that it employs pre-computed, three-sentence extractive summaries generated using LexRank and TextRank algorithms with the objective to reduce computational overhead during the Transformer inference phase.

4.2.3 Data Engineering and out-of-core processing

The primary dataset is the FNSPID. A very large repository that consists of 15.7 million financial records, this means that loading a dataset of this magnitude that exceeds 22GB in raw CSV format into a standard virtual machine would trigger an out of memory collapse. An out-of-core extractive sampling methodology was applied, which refers to techniques that allow algorithms to process a massive quantity of data and make it fit into a computer's RAM(Schrádi & Juhász, 2009; Out of Core Computation — Strax 2.2.2 Documentation, 2018)

In order to process data, a data chunking methodology was applied to reach this objective. This processing strategy breaks large files into smaller and sequential partitions that facilitate fitting in RAM. To ensure its viability, calculations are performed on each chunk, and then the results are aggregated (Schrádi et al., 2011). In this context, the FNSPID was scanned sequentially in chunks of 500,000 rows to keep the RAM under 1kB during the scan. Then, the algorithm aggregated the news frequency for the entire dataset. This Exploratory Data Analysis was applied to map publication volumes and target only those stocks with high informational density. During this first scan, the algorithm identified 20 tickers with a high density of news. However, some highly mentioned entities were not stocks or are not useful for this model, for

example, Brooge Energy Limited (BROGW) was a company that offered oil storage and blending services based in the United Arab Emirates, but retired from the Nasdaq stock exchange following an SEC investigation that exposed a massive accounting fraud involving inflated revenues (U.S. Securities and Exchange Commission, 2023). Another example of an unhelpful stock is Brookfield Property Partners L.P. (BPYPO). Although it is a real estate company, the BPYPO ticker represents its Series 2 cumulative preferred shares, which differ from standard equity shares because they function like fixed-income securities that pay regular dividends (Yahoo! Finance, n.d.).

These non-stock instruments, as well as Exchange Traded Funds (ETFs) and market indices, were filtered to preserve the purity of the common stock cluster. This procedure led to the identification of six high-density news stocks, compounded by Apple Inc. (AAPL), Microsoft Corporation (MSFT), NVIDIA Corporation (NVDA), Tesla, Inc. (TSLA), Alphabet Inc. (GOOG), and Intel Corporation (INTC). In addition, Amazon.com, Inc. (AMZN) was added to complete the major technology and semiconductor sector. Finally, this selection resulted in a big tech cluster of seven stocks that contain 64,842 continuous news records over the 2018 - 2023 period, which averages approximately 9,263 articles per asset that ensures a robust size for the deep learning layer.

4.2.4 Deep Learning and Sentiment Extraction (FinBert)

Financial texts might be challenging to classify sentiment, especially in general-purpose Natural Language Processing (NLP) models, because financial words are often dependent on context. To solve this issue, the Financial Bidirectional Encoder Representations from Transformers (FinBert) was applied due to its capacity to solve this limitation. This framework is continued pre-trained on a massive quantitative financial corpus of 4.9 billion tokens. The FinBERT model was loaded with a batch size of 32 to maximize parallelization, and it generated raw predictions for each headline and classified it as positive, negative, or neutral. It was necessary to apply a Softmax activation function[2] to obtain normalized probability distributions from these labels, instead of utilizing categorical labels, a continuous sentiment score was calculated (Equation 1):

[2] Deep learning systems depend on activation functions, which play an important role because they allow for the necessary non-linearity in expressing complex, high-dimensional patterns. In particular, the softmax function is applied to the output layer of multi-class classification systems to process the vector of any real-valued scores (logits) and obtain the probability distribution corresponding to the output scores. Softmax, therefore, guarantees that the output values lie in the interval [0, 1] and the output values sum strictly to 1 (Goodfellow et al., 2016).

$$SentimentScore = P(positive - negative) \quad (1)$$

The sentiment score ranges between -1.0 (extremely bearish) and +1.0 (extremely bullish), which represents all the informational consensus of news daily. To avoid look-ahead bias and ensure predictive validity, all intraday sentiment scores were added per asset day using the arithmetic mean and applied a 1-day temporal lag (Day T -1), this means that a sentiment known on Monday (Day T -1) turned into an indicator for the market on Tuesday (Day T), serving as a trigger or signal of a position opened at the Tuesday closing price, which reflects a realistic scenario where an investor reads and analyzes the news overnight to place orders the following day (in this case in the closing price).

4.2.5 Features and variable engineering

To build the predictive model with important parameters and prevent data leakage, some variables were created using out-of-sample past values to avoid look-ahead bias, thereby representing various dimensions of asset risk, macroeconomic conditions, and momentum.

a. **Lagged FinBERT Sentiment (Lagged_Sentiment_Tminus1)**

   The FinBERT model processes daily financial news on the T4 GPU. Following the methodology established by Tetlock (2007) to capture lagged market reactions. The daily score is calculated as the mean difference of raw softmax probabilities of being positive versus all negative news published during day T-1, as illustrated in Equation 2, this signal, between -1 and +1, provides a signal to identify the investor sentiment:

$$Lagged - Sentiment - Tminus1_t = \frac{1}{N_{t-1}} \sum_{i=1}^{N_{t-1}} (P(positive)_{i,t-1} - P(negative)_{i,t-1} \quad (2)$$

   $N_{t-1} =$ Represents the total of news processed in the previous day (Day T-1) on day T.

   $P(positive)_{i,t-1}$ and $P\ (negative)_{i,t-1} =$ Capture the net sentiment polarity from the previous day.

b. **Lagged Chicago Board Options Exchange (CBOE) Market Volatility Index (VIX). Volatility Index (VIX_Lag1)**

The CBOE Volatility Index is included as a macroeconomic indicator used to assess market sentiment and system-wide risk. Whaley (2009) defines the VIX as the market's expectation of 30-day forward-looking volatility derived from a wide cross-section of S&P 500 index options. In order to enable the model to identify shifts in global market risk without data leakage, one-day lagged value of the VIX is used as a predictor (see Equation 3), the model can identify global market risk and decrease capital exposure $VIX\ LAG1_t > 25$ to ensure capital preservation.

$$VIX\ -\ LAG1_t = VIX_{t-1} \tag{3}$$

c. Historical Volatility (Vol_20d)

Historical Volatility (HV) is an indicator that measures the price fluctuations of an asset over time to identify the associated risk. It is defined as a statistical measure representing the dispersion of an asset's or index's returns over a specified time horizon. In quantitative finance, it is typically calculated as the annualized standard deviation of the asset's continuously compounded (logarithmic) daily returns (Hull, 2022). In this context, it follows Hull´s procedure to estimate volatility from historical data (2021) and is calculated as the standard deviation of daily log returns over a rolling 20-day window and it is also rescaled to the annual trading year as it shows in Equation 4 and 5:

$$Vol20d_t = \sigma_{ann} \cdot \sqrt{\frac{1}{n-1}\sum_{i=0}^{n-1}(r_{t-1} - \bar{r})^2} \tag{4}$$

$$Vol20d_t = \sqrt{252} \cdot \sqrt{\frac{1}{19}\sum_{i=0}^{19}\left(ln\left(\frac{Close_{t-i}}{Close_{t-i-1}}\right) - \bar{r}\right)^2} \tag{5}$$

Where:

σ_{ann} : Represents the annualization factor, calculated to adjust the volatility to an annual basis, based on trading days.

r_{t-1}: Is the log-return of the asset in period t-i, calculated as $\frac{Close_{t-i}}{Close_{t-i-1}}$.

n: Is the number of observations in the rolling window. In this context, it is 20.

d. Relative Strength Index (RSI_14)

The Relative Strength Index (RSI_14) is defined as a momentum indicator used in technical analysis that quantifies the velocity and magnitude of directional price movements. Operating on a normalized scale from 0 to 100, this metric is primarily utilized to identify statistically overextended market regimes (Wilder, 1978). Values above 70 indicate overbought conditions in the ticker, while values below 30 show oversold levels. This feature is essential because it prevents the model from executing buy orders on positive news if the stock is already expensive, as it is overbought. In this context, the RSI is calculated, following Wilder's 1978 methodology, over a 14 day lookback window using Exponential Moving Averages (EMA) of price gains (U) and losses (D), as it can be illustrated in Equation 6:

$$RSI14_t = 100 - \left(\frac{100}{1+RS_t}\right) where\ RS_t = \frac{EMA(U,14)_t}{EMA\ (D,14)_t} \quad (6)$$

Where:

Relative Strength RS_t: It is calculated as the ratio between the average gains and the average losses

$EMA(U,\ 14)_t$: Exponential Moving Average of positive price changes (gains) over the last n periods

$EMA\ (D,\ 14)_t$: Exponential Moving Average of negative price changes (losses) over the last n periods.

e. Tickers as categorial asset Encoders (Ticker_Code)

A label encoding vector is included as Ticker_Code $\in \{0, 1, 2, 3, 4, 5, 6\}$ to identify each stock, such as AAPL, AMZN, GOOG, INTC, MSFT, NVDA, and TSLA, respectively. This is necessary so the XGBoost can distinguish asset specific patterns and behaviour

(since stocks may behave differently according to the beta[3] associated with them) without treating the tickers as continuous numerical values.

4.2.6 Triple Barrier Labeling (TBL) Method as a dynamic target labeling framework

The Triple Barrier Labeling method was "first introduced by de Prado, is a widely used method for generating labels in financial time series prediction by incorporating stop-loss, take-profit, and time horizons" (Kang, 2025, p.2). According to Kang, TBL has outperformed traditional labeling techniques such as fixed time horizon (FTH) and raw return (RR), specifically in identifying buy signals, and it is widely used to achieve balanced label proportions. The TBL was implemented in order to train the supervised machine learning method under realistic risk management conditions, which consisted in three barriers around each daily closing price ($Close_t$):

1. Upper barrier: This barrier triggers a profit taking with a label of +1 in case this objective price is reached.
2. Lower barrier: It triggers a stop loss exit to protect capital with a label of -1 if it is breached.
3. Vertical barrier: It is utilized when time expires. This means that it triggers a time based position liquidation with a label of 0 if the price does not touch either horizontal barrier within a 5 day time horizon.

Rather than using a static percentage, the horizontal barriers adjust dynamically according to each asset's 20 day annualized volatility (Vol_20d) rescaled to the 5 day vertical holding time horizon. Daily log returns were computed from Adjusted Close prices downloaded via yfinance to calculate the rolling standard deviation.

According to López de Prado (2018) this method defines the barriers based on the asset's return over a given period. The methodology to calculate it is defined $r_{t_0,t}$ as the return from the entry time t_0 to a future time, defined in Equation 7 as:

[3] Beta is an indicator of a stock's price volatility compared to the market, the higher the beta number, the higher the risk (Kenton, 2026).

$$r_{t0,t} = \frac{Pt}{P_{t_0}} - 1 \quad (7)$$

According to López de Prado, the horizontal barriers are triggered under the following conditions:

- Upper barrier: Occurs when $r_{t_0,t} > pt \cdot \sigma_{t_0}$
- Lower barrier: Occurs when $r_{t_0,t} \leq sl \cdot \sigma_{t_0}$ (8)

Where:

$r_{t0,t}$: Is the calculated return of the asset between entry and the current observation.

t_0: The day where the position is initiated.

t: Represents the future day being evaluated.

t_0: Is the initial execution price of the asset.

Pt: Is the intraday price being evaluated for a barrier touch.

pt and sl: Are the asymmetric multipliers for profit taking and stop loss.

σ_{t_0} :Represents the estimated daily volatility at the time of entry.

For this project the barriers were calculated following an asymmetric risk ratio 2:1 as the multipliers for profit taking and stop loss as it can be seen in the Equations 9 and 10:

$$UpperBarrier = Close_T \cdot \left(1 + \left(2 \times \frac{Vol20d_t}{\sqrt{252}}\right)\right) \quad (9)$$

$$LowerBarrier = Close_T \cdot \left(1 - \left(1 \times \frac{Vol20d_t}{\sqrt{252}}\right)\right) \quad (10)$$

In this context, the algorithm executed a path dependent for 5,178 trading days forward search over the subsequent 5 trading sessions, which also checked the intraday high and low values against these dynamic barriers.

4.2.7 The XGBoost Algorithm

The eXtreme Gradient Boosting (XGboost) is "a distributed, open-source machine learning library that uses gradient boosted decision trees, a supervised learning boosting algorithm that makes use of gradient descent. It is known for its speed, efficiency and ability to scale well with large datasets" (IBM, 2024, par. 2) and it is designed for structured and tabular data. This algorithm was selected because of many advantages it offers when processing noisy and non stationary financial features.

First of all, intending to predict financial markets using technical indicators and sentiment from financial texts is highly challenging because these kinds of datasets usually contain a grand quantity of noise that leads to unuseful data and financial markets usually tend to have unpredictable and non stationary conditions. Which means that, XGBoost is excellent for this since it handles diverse types of data with different scales and categories much better natively, without requiring complex data transformation or extensive normalization. While Deep Learning Models (like Long short-term memory (LSTM)[4]) are very good at identifying long-term historical trends from high-frequency price data but tend to fail due to market noise and also frequently suffer from severe overfitting because they memorize past noise instead of learning patterns that work on new, different, and unseen data and demand a great quantity of computing power. (Reddy, 2025; Fischer, T., & Krauss, C. 2017)

Another important benefit of the algorithmic architecture of XGBoost is that it incorporates L1 (Lasso) and L2 (Ridge)[5] parameters for regularization within the loss function to penalize model complexity and prevent individual trees from memorizing market noise or overrelying on a dominant feature. Also, XGBoost is an excellent selection as it focuses on correcting the residual errors from previous trees that lead to highly robust generalized out of sample predictions. (Priya et al., 2024)

XGBoost provides a feature importance hierarchy that is based on the decision weight (F-score), which allows the model to be transparent and comply with explainability and

[4]A long short-term memory architecture (LSTM) is a type of recurrent neural network (RNN) that is designed to learn and remember information over long sequences of data (Noble, 2025).

[5] Ridge and Lasso Regression are methods in machine learning used for regularizing linear models with the objective of avoiding overfitting and improving predictive performance. Both methods add a penalty term to the model's cost function to constrain the coefficients, but they differ in how they apply this penalty. Ridge adds the squared magnitude of the coefficients as a penalty and Lasso introduces a penalty based on the absolute value of the coefficients (Hastie et al., 2009).

transparency standards, like the European MiFID II directive, and ensure that investment decisions can be revised and audited (Fu et al., 2026; Xiao, 2025). The classifier was configured with the following parameters to prevent overfitting:

1. Max_depth = 5: Limits the depth of individual trees to promote better generalization.
2. N_estimators = 150: Sets the number of boosting iterations to ensure complete and robust learning.
3. Learning_rate = 0.03: Controls the learning step size for a gradual and stable gradient descent.
4. Subsample = 0.8 and colsample_bytree = 0.8: Introduces stochastic regularization by selecting 80% of rows and columns per tree, ensuring that all hybrid variables actively contribute to the model.

4.2.8 The Kelly Criterion for portfolio capital allocation and position size per trade

The Kelly Criterion is utilized to maximize the expected value of "the logarithm of wealth ("maximize expected logarithmic utility"). The criterion is known to economists and financial theorists by names such as the "geometric mean maximizing portfolio strategy", maximizing logarithmic utility, the growth-optimal strategy, the capital growth criterion, etc" (Thorp, 2006). This model maximizes wealth growth by determining optimal investment sizes, considering winning probability and win and loss ratio (Kelly, 1956), however, It is important to consider that the Kelly criterion aims to maximize the expected geometric mean return of the portfolio without considering risk factors, which makes this strategy very aggressive (Xing, 2025), which explains why in this model, only a fractional Kelly approach was applied using a multiplier of 0.60 to reduce excessive volatility and over exposure.

To manage risk and avoid concentration on a specific asset, a maximum allocation constraint of 35% of total portfolio capital was applied. The size of the portfolio allocation is dynamic because it scales proportionally with the model's confidence as it predicted is by XGBoost probability (P_{XGB}), the calculations of the position size can be illustrated in the following Equations 11 and 12 (Thorp, 2006; Kelly, 1956):

$$f_{kelly} = \frac{bp-q}{b} \quad (11)$$

$$Position\ Size = min\left(Capital \times \left(0.60\ f_{kelly}\right),\ Capital \times 0.35\right) \times P_{XGB} \quad (12)$$

Where:

f_{kelly}: The fraction of the total capital to be allocated to the trade.

b: The ratio of the potential gain to the potential loss.

p: The probability of winning.

q: The probability of losing (which is calculated as $1 - p$).

$Position\ Size$: The final capital amount allocated to the specific trade.

$Capital$: The total available cash of the portfolio capital.

0.60: The fractional Kelly multiplier used to mitigate volatility.

0.35: The constraint to limit capital allocation to a maximum of 35% of the total capital per ticker.

P_{XGB}: The predicted probability of success from the XGBoost model.

4.2.9 Portfolio Backtesting Engine

The final step is the Simulation Engine, which was built directly in Python to ensure transparency and auditability (Hilpisch, 2018). The simulation was configured with a constraint of initial capital of $100,000.00 and the strategy was very related to the end of the day (EOD) closing price (day t) using the information from the day before (day t-1). This prediction strategy was adapted because the system processes consolidated data after the close of each trading session (including sentiment analysis, technical, and volatility indicators from the day before day t, day t-1). In other words, prescriptive signals are generated using data through the close of day t-1, position openings are executed at the closing price of the subsequent session (day t), this strategy is strong due to the following reasons:

- **Consistency:** Executing at the closing price eliminates the stochastic ambiguity inherent in intraday trading related to the noise in the financial markets (Bäuerle &

MacKay, 2026). This ensures that the backtest remains temporally aligned with the historical prices used to calculate technical indicators.

- **Noise Mitigation:** The strategy avoids false breakouts during the highly volatile opening session. It focuses on stabilized closing trends.
- **Temporal Synchronization:** The technical indicators used (RSI_14 and Vol_20d) are calculated using daily closing prices, this means that, entering to the market at the close of the day t maintains a temporal alignment between the features and the entry point.

In order to make the backtester model realistic, a cost of 0.20% per transaction per execution and a 0.03% slippage penalty to account for execution decay at the close were applied following the market friction parameters described by Narang (2013). If a buy signal matched a take profit objective, the system recorded this gain minus the costs. However, if it reached a stop loss, it recorded the loss plus the costs. On the other hand, if any of the boundaries were reached, the position was closed at the closing price of day 5. The resulting Profit and Loss (PnL) is then calculated as the percentage price delta between the entry price on day t and the liquidation closing price on day t+5 minus transaction costs.

With the objective to compare how profitable the model is, the performance of the portfolio was compared to a passive Buy & Hold strategy of the S&P 500 Index (SPY)[6] aligned with the exact out-of-sample timeline. The following Key Performance Indicators (KPIs) were calculated to measure the model´s performance:

- **Cumulative Return (ROI):** The total net profit or loss generated by the strategy relative to the initial $100,000 capital investment.
- **Jensen's Alpha (α):** It is a financial metric used to evaluate how well an investment performs against the market, adjusted for its systematic market risk (Jensen, 1968)..

$$\alpha = Rp - [Rf + \beta \times (Rm - Rf)] \quad (13)$$

Where:

[6] The S&P 500 (Standard & Poor's 500) is a market-capitalization-weighted index of 500 of the leading publicly traded corporations in the US. It is used as an indicator of the overall health of the US stock market and the broader economy. The PDR S&P 500 ETF Trust (known as the SPY) is an Exchange-Traded Fund (ETF) that tracks the S&P 500 Index. (State Street Global Advisors, 2026).

Rp : The realized return of the portfolio.

Rm: The realized return of the market index.

Rf: The risk-free rate.

β: The beta of the investment portfolio related to the chosen market index.

- **Sharpe Ratio:** It is a performance measure first introduced by William F. Sharpe in 1994, which measures risk-adjusted performance and computes how much excess return an investment has generated over a risk-free rate for every unit of total risk (standard deviation). It is defined as the ratio of the annualized excess return to the standard deviation of portfolio returns.

$Sharpe\ ratio = \frac{Rp - Rf}{\sigma_p}$ where σ_p is the standard deviation of the portfolio´s return. (14)

- **Sortino Ratio:** It is very related to the Sharpe ratio. In this context, this metric provides a more nuanced evaluation of risk-adjusted performance by addressing the limitation of penalizing upside volatility. Instead of utilizing the total standard deviation of portfolio returns, it isolates downside risk by exclusively measuring the standard deviation of negative returns—known as downside deviation—thereby providing a more precise assessment of capital at risk (Sortino & Price, 1994).

$Sortino\ Ratio = \frac{Rp-rf}{\sigma_d}$ where σ_d is the standard deviation of negative returns. (15)

- **Maximum Drawdown (MDD):** It is defined as the largest peak-to-trough drop in the value of a portfolio or asset before a new peak is achieved. MDD serves as a primary measure of historical downside risk, illustrating the maximum capital loss an investor could have experienced during a specific period (Bacon, 2023).

$$MDD = \frac{Trough\ Value - Peak\ Value}{Peak\ Value} \quad (16)$$

- **Calmar Ratio:** This indicator is used to evaluate the performance and evaluates the relationship between the annualized return of a portfolio and its maximum drawdown over a specified period. By penalizing returns strictly based on the maximum drawdown rather than general volatility, the ratio effectively quantifies the strategy's ability to generate yield relative to its extreme downside risk exposure (Bacon, 2008).

$$Calmar\ ratio = \frac{R_a}{MDD} \text{ where } R_a \text{ is the annualized return.} \quad (17)$$

- **Profit Factor:** This metric calculates the ratio of gross gains to gross losses, which helps to understand if the model is profitable due to it measures how much profit a trading strategy generates for every unit of loss it incurs. A value above 1.0 indicates a profitable strategy (TradingView, 2026).

$$Profit\ Factor = \frac{\sum Gross\ Profit}{\sum Gross\ Losses} \quad (18)$$

- **Information Ratio (IR):** This indicator is essential because it "measures a fund manager's returns against a benchmark and the volatility, or consistency, of those returns (Gratton, 2025, par. 3), which represents the active return of the portfolio divided by the tracking error (the volatility of the excess returns). It is crucial because it helps to validate that the strategy is not just beating the market by chance, but doing so through consistent, risk-controlled alpha generation.

$$IR = \frac{Portfolio\ Return - Benchmark\ Return}{Tracking\ Error} \quad (19)$$

4.2.10 Practical example and investor operational rules

To illustrate the deployment and how the system works, a daily operation framework from the perspective of an institutional investor or quantitative trading desk is established. The system operates under an end of the day strategy that follows the next five steps:

1. **Unstructured data collection:** Every day, after the market closure at 16:00 EST, the system collects all financial news headlines related to the stock market, specifically the Nasdaq (National Association of Securities Dealers Automated Quotation)[7] published over the preceding 24 hours (day t-1). These headlines are compiled so the FinBERT calculates the sentiment score for each asset in the cluster selected.

[7] The Nasdaq is the second-largest stock exchange globally by market capitalization and operates as a fully electronic trading platform. Headquartered in New York City, it facilitates the trading of over 5,000 listed equities. The exchange is heavily weighted toward technology and growth-oriented firms, serving as the primary listing venue for mega-cap technology corporations such as Apple, Microsoft, and Amazon (Bodie et al, 2024).

2. **Feature Synchronization:** At the same time, the system downloads the quantitative market data to calculate the quantitative indicators. To prevent look-ahead bias and maintain temporal synchronization with the stock sentiment score, all technical and macroeconomic features, such as, daily closing price (P_{t-1}), the 14-day Relative Strength Index (RSI_14), the 20-day annualized rolling volatility (Vol_20d), and the CBOE Volatility Index (VIX_{t-1}), are calculated using historical data up to the day t-1.

3. **Decision Rules test:** In this step, the model calculates the probability distribution for the three potential predictive outcomes: Buy (P_{Buy}), Sell (P_{Sell}), and Hold (P_{Hold}):

    a. **Buy signal (P_{Buy} > 0.42**): A position is opened (+1) if the predictive confidence exceeds 42% because it is above the 33% baseline to prioritize precision rather than recall. This means that it ensures that capital is deployed only when the confluence of NLP sentiment and momentum exhibits a statistically significant edge over market noise

    b. **Defense to protect capital preservation (P_{Sell} > 0.45)**: A Sell (-1) signal does not initiate a short position. Instead, it identifies bull markets and behaves as a filter that blocks any new capital deployment to hold available equity in risk-free cash. It has to be higher of 0.45 to provide significant evidence of pessimism over the stock to activate the defense mechanism, preventing over-reactions to standard volatility, in this way, the system manages risk and ensures capital preservation.

    c. **Default (Hold):** If neither the Buy nor the Sell confidence thresholds are met, the system prescribes a "Hold" (0) to avoid ambiguous markets and private capital for a preciser opportunity.

4. **Execution:** If a buy signal is triggered, the system calculates the optimal position size utilizing a Fractional Kelly Criterion (f = 0.60), buys at the closing price of day t and calculates the available cash and ensures that the constraint of a maximum of 35% of the total available capital per trade is not violated to prevent over-leverage on a single operation.

5. **Monitoring:** Once the operation is opened, it enters into a path-dependent monitoring cycle where the position is liquidated if it reaches Take-Profit (PT) or Stop-Loss (SL) boundaries, or it is closed at the closing price of day t+5.

4.3 Required Resources

The execution of this project required several technical and computational resources, which are detailed below:

Technical Resources:

- Programming Language: Python.
- Software Frameworks and Libraries:
    - XGBoost (for the gradient-boosted classification engine).
    - Hugging Face Transformers (for FinBERT sentiment analysis).
    - Pandas and NumPy (for ETL, data chunking, and vectorization).
    - Scikit-learn (for classification diagnostics and simple weighting).
    - Matplotlib and Seaborn (for equity curve and visualizations).
    - Yahoo Finance API (for market data acquisition).

Computational Infrastructure and Devices:

- Google Colab: Cloud-based Integrated Development Environment (IDE).
- NVIDIA T4 GPU: Hardware acceleration for deep learning inference and matrix operations.
- Google Drive: Cloud storage for data lineage and asset serialization.

Data Resources:

- Financial News and Stock Price Integration Dataset (FNSPID): The primary qualitative corpus, providing 15.7 million historical financial news records utilized for sentiment extraction.

- Yahoo Finance API (yfinance): The quantitative data provider used to extract daily Open, High, Low, Close, and Volume (OHLCV) market data, as well as macroeconomic indicators (VIX).

4.4 Budget

Table 1. Budget

Cost Type	Estimated Value (€)	Comments
Project Working Hours	450 h	Total estimated effort dedicated to literature review, data processing, sentiment analysis, feature engineering, model development, validation, documentation, and thesis writing.
Academic Supervision	80 h	Estimated supervision, review meetings, methodological guidance, and evaluation support provided by thesis supervisors.
Technical Equipment Used	1,500 €	Personal laptop/workstation used for data processing, NLP inference, model training, and visualization tasks. Estimated market value if acquired new.
Cloud / Computational Resources	0 €	No external cloud computing infrastructure was contracted. Processing was performed locally.
Software Used	0 €	Open-source software and libraries were used, including Python, Jupyter Notebook, Pandas, NumPy, Scikit-learn, XGBoost, Hugging Face Transformers, Matplotlib, and Seaborn.

Cost Type	Estimated Value (€)	Comments
Data Sources	0 €	Financial news datasets and historical market data were obtained through publicly accessible sources and APIs such as Yahoo Finance and Hugging Face Datasets.
Studies and Reports	0 €	Academic publications and technical references accessed through university databases and open-access repositories.
Materials Used	0 €	No physical laboratory materials or sensors were required for the development of the project.
Internet and Communication Resources	200 €	Estimated proportional cost of internet services and online collaboration resources used throughout the project duration.

4.5 Feasibility

The technical and economic feasibility of the project is primarily driven by its usage of open source tools and publicly available datasets. Most used tools in this project such as Python, Jupyter Notebook, Scikit-learn, XGBoost, and FinBERT are free and well-supported by both academia and industry. Furthermore, financial news data and historical market data were obtained from accessible repositories and APIs.

From an economic point of view, the project was implemented with limited investment because the only relevant resources are computing power, time, and existing hardware. All the costs associated with the project relate to the time spent on data processing, feature engineering, model development, and validation processes. Therefore, costs in their relationship with benefits seem to be satisfactory, due to the big potential benefits that can be obtained from combining sentiment analysis with quantitative indicators.

The proposed system is a limited minimum viable product that allows for making the project sustainable in the long term due to its scalability and flexibility. The methodology can be applied to new assets, financial markets, and data sources without major infrastructure changes. In addition, by means of the use of modular machine learning pipelines and automation processes, the project can be maintained and upgraded in the future with the use of APIs for real time information like news from other sources and stock prices, requiring much more computational resources.

Another important point for sustainability is the reproducibility of the methodology. Thanks to the use of publicly available tools, libraries, and standard research methods, the project can be escalated by other researchers interested in the area of financial sentiment analysis and machine learning in finance. The analysis proves that the project is feasible from both technical and economic points of view.

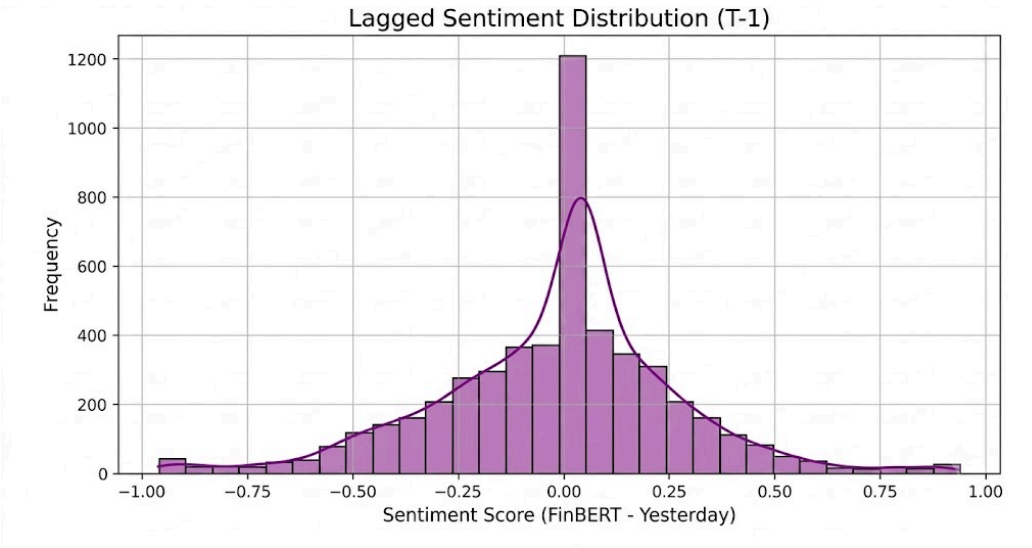
4.6 Project results

In this section the results are presented and structured around the study's primary objectives: Quantify unstructured market sentiment, develop and train a machine learning-based classification engine (XGBoost), and evaluate the model performance. Before introducing the predictive model, the unstructured financial news corpus processed by FinBERT is verified and aligned with the other technical and macroeconomic indicators, to ensure that it provides a robust foundation for optimizing investment decision-making.

4.6.1 Alternative data validation and sentiment distribution

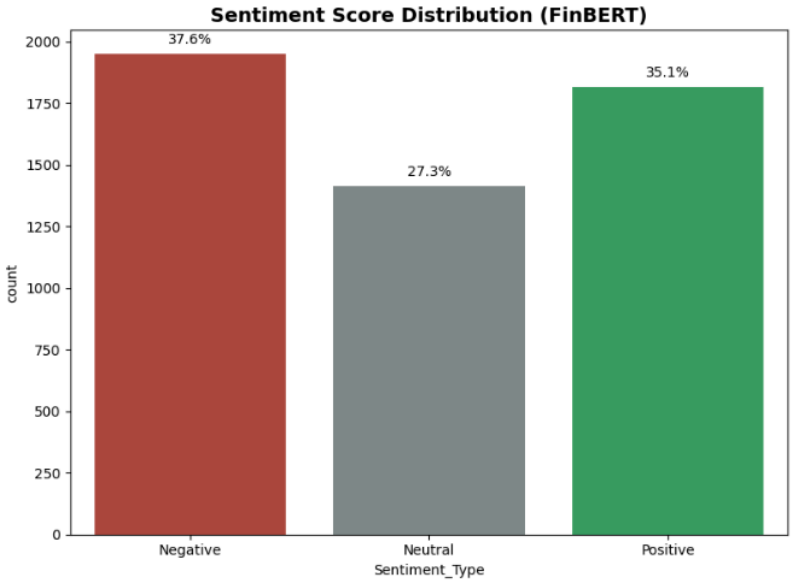
In order to validate the natural language processing (NLP) pipeline, the first step is to assess the general distribution of all the quantified sentiment scores in the total processed dataset. With financial NLP, pre-trained bots usually classify a majority of cases as neutral (with a sentiment score of zero), leaving the entire dataset without any predictive variance to help a trained model create accurate predictions (Araci, 2019; Loughran & McDonald, 2011). As Lopez de Prado (2018) states, when financial features lack sufficient statistical variance, machine learning algorithms cannot identify significant patterns, resulting in uninterpretable and non-functioning data for predictive modeling. Therefore, it is crucial to confirm that the sentiment scores are properly distributed to provide the XGBoost engine with a reliable signal to utilize for calculations.

Figure 2. Lagged Sentiment Distribution (T-1)



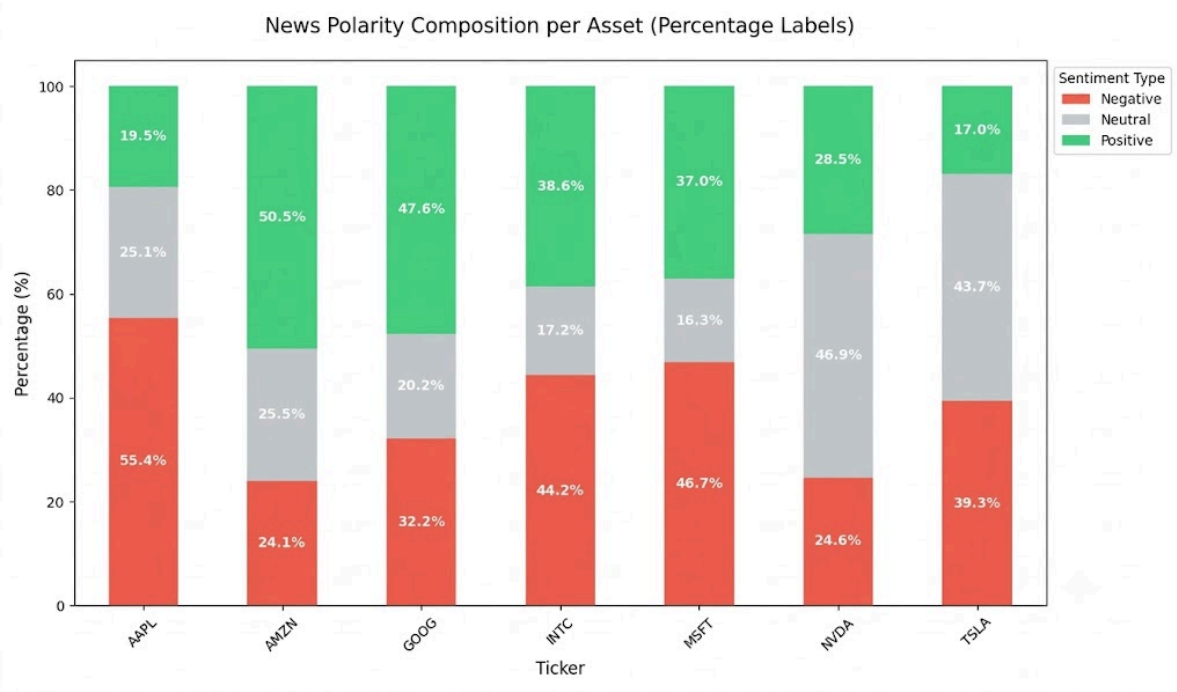
The Lagged Sentiment Distribution (T-1) histogram shows that the FinBERT generates an output that does not have this issue. The output signal has a quasi-symmetrical leptokurtic distribution rather than following a common normal (Gaussian) distribution; there is a large expected structural peak around absolute neutrality (0.0), and the signal has huge variance with the fat tails capturing extreme polarity across the entire -1 to +1 range, which is an accurate representation of the non-common nature of financial market shocks. In practical terms, this means that most of the daily financial news is neutral in nature (the central peak); however, the model identifies and isolates true positive and negative sentiment drivers without simply defaulting to the mean.

Figure 3. Sentiment Score Distribution using FinBERT



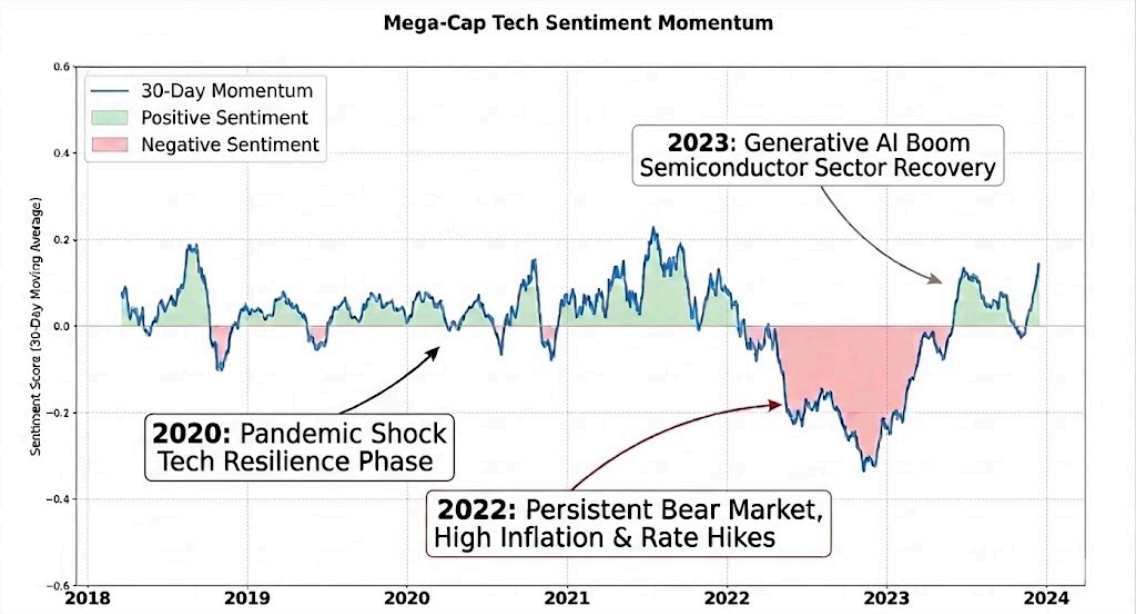
This continuous variance seen previously in figure 2 is proven with the categorical Sentiment Score Distribution (Figure 3). The corpus has been classified as negative (37.6%), neutral (27.3%), and positive (35.1%). This shows that the model has a balanced polarization, thereby confirming that FinBERT possesses sufficient sensitivity to capture both pessimistic market sentiment and optimistic breakthroughs while avoiding the tendency to simply revert to the mean. This finding is significant because it has achieved a balanced polarity. Ensuring that the Neutral class remained the minority class, guaranteeing that FinBERT's context awareness is robust enough to confidently identify negative and positive sentiment.

Figure 4. News Polarity Composition per Asset (Percentage labels)



After validating the global corpus sentiment score distribution. It is interesting to drill down into the granular distribution of sentiment across the Big Tech cluster. As shown in the News Polarity Composition per Asset (figure 4), the global variance is distributed effectively per asset rather than giving a uniform distribution to all the stocks. Actually, the sentiment framework successfully differentiates between entities and identifies the reality of each company, which confirms that the NLP model captures drivers per ticker, ensuring that the algorithmic framework will be able to distinguish and isolate specific, risk optimized asset opportunities rather than only tracking the broader macroeconomic trend.

Figure 5. Mega - Cap Tech Sentiment Momentum



It is crucial that alternative data reflects the broader macroeconomic environment over continuous timeframes accurately. In order to evaluate its consistency, a 30-day moving average of the aggregate sentiment score was mapped across the entire out-of-sample timeline. As it its illustrated in the Mega - Cap Tech Sentiment Momentum Chart (figure 5), the quantified FinBERT sentiment score exhibits a high degree of temporal accuracy, aligning precisely with historical macroeconomic stress events and market cycles:

- **Pandemic (2020):** The signal reflects the initial panic driven drop followed by a rapid recovery. The early panic driven decline in Mega-Cap tech was replaced by the post-pandemic increase in value due to the growth of E-Commerce, Cloud Computing, and Remote-work (Tadrzak, 2020).
- **Persistent Bear Market (2022):** The extended negative trough represents months of decline that was driven by macro pessimism due to Federal Reserve Rate increases, Silicon Valley mass layoffs, wide drawdowns in sectors, and systemic recession fears (Randall, 2022).
- **Artificial Intelligence Boom (2023):** The chart shows a sharp, V-shaped breakout over the last ten months due to the increase in public euphoria for new generative A.I. Models, along with an increase in demand for semiconductors caused by generative

> A.I. technology launching more consumer products than all consumer product launches combined over the last century (Lauricella & Schlueter, 2026).

The empirical evidence of the global distribution, idiosyncratic per-asset variance, and macro-momentum temporal alignment verifies that the first research objective has been fully met. The FinBERT framework has effectively extracted a non-stationary, highly contextual sentiment signal derived from unstructured media. Therefore, this indicates that the engineered dataset is not random noise, and it is a data matrix fully prepared for ingestion by the predictive classification engine.

4.6.2 Predictive Engine Development and Signal Translation

The construction of the multimodal dataset was followed by the generation of volatility-adjusted labels using the Triple Barrier Labeling methodology. The development of a predictive engine then enabled sentiment and market information to be transformed into actionable investment signals.

The subsequent development of a predictive engine then enabled sentiment and market information to be transformed into actionable investment signals. The XGBoost classifier was selected on the basis of its proven performance in handling complex, non-linear relationships and heterogeneous data sources.

The model was trained using a combination of FinBERT-derived sentiment indicators and quantitative market features, including volatility, momentum, and trading activity measures. In addition to the predictive performance of the model, emphasis was placed on the interpretability of the model to evaluate the contribution of individual features and to understand how sentiment information influences investment decisions.

The following figures assess the predictive engine from complementary perspectives, including classification performance, feature importance, and the translation of sentiment signals into both alpha-generation and risk-management mechanisms.

Table 2. Classification Report (Test set)

Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
Sell (-1)	0.61	0.23	0.33	987
Hold (0)	0.18	0.79	0.29	246
Buy (1)	0.41	0.28	0.33	682
Accuracy			*0.32*	*1915*
Macro Avg	**0.40**	**0.43**	**0.32**	**1915**
Weighted Avg	**0.48**	**0.32**	**0.33**	**1915**

As illustrated in table 2, the classification report obtained from the XGBoost model on the test dataset that was not used to train the model is presented. The present report is an evaluation of the predictive performance of the classifier through precision, recall, F1-score, and overall accuracy metrics for the three classes generated by the Triple Barrier Labeling (TBL) framework. The recommended course of action is to purchase (1), maintain a hold on the asset (0), or divest (-1).

The most significant discovery that emerged from the testing dataset is related to the model's structural asymmetry in identifying risks. In fact, the classifier's Precision level was an impressive 0.61 (or 61%) with respect to the negative signal (Sell or -1). Therefore, every time the algorithm generated a Sell signal (or negative directional signal), it was correct 61% of the time when predicting systemic market fear or contraction. By contrast, the model has proven to be more conservative than expected, successfully identifying only 79% (Recall = 0.79) of all nondirectional periods (Hold or 0) as an accurate prediction. This demonstrates that the algorithm does not function in a traditional manner as an equally balanced classifier; instead, it works more conservatively than is typical and serves primarily as an asymmetrically conservative risk-filtering mechanism.

The findings suggest that the model demonstrated the capacity to discern significant patterns within the integrated feature set, which encompassed sentiment, macroeconomic, and technical indicators. The classification performance demonstrates that the predictive engine successfully transformed unstructured financial news and market information into actionable investment signals. The employment of XGBoost, in conjunction with the volatility-adjusted TBL

methodology, has been demonstrated to contribute to the reduction of noise and the enhancement of class separation in a highly non-stationary financial environment.

From a pragmatic standpoint, the classification report substantiates the viability of incorporating sentiment information into financial prediction tasks. Rather than relying exclusively on price-based indicators, the model utilises complementary information derived from financial news, thereby enhancing its capacity to differentiate between favourable, unfavourable, and neutral market conditions. The findings of this study lend support to the hypothesis that sentiment contributes incremental predictive value when integrated within a multimodal financial framework.

Figure 6. Algorithmic Feature Hierarchy: What Drives the Model?

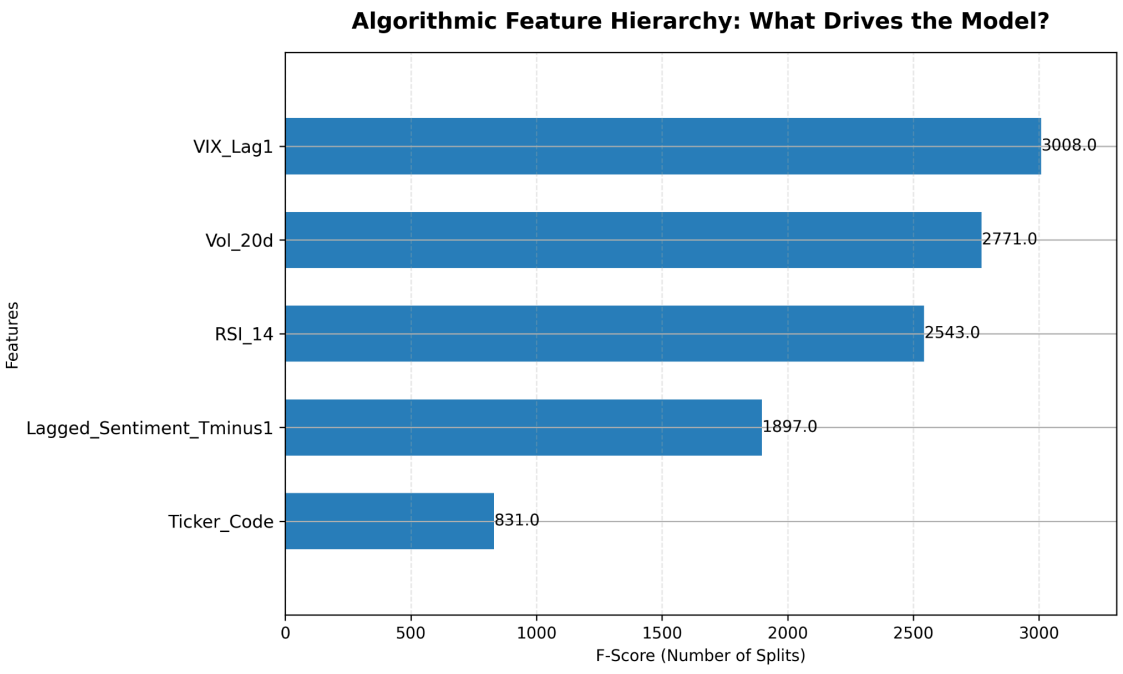


Figure 6 illustrates how the feature importance hierarchy created by the XGBoost classifier enhances the clarity of the internal decision-making hierarchy used by the XGBoost Algorithm to create Investment Signals (IS). By displaying how important the various variables are, this hierarchy displays transparency into the relative strength of each feature's influence against other features on an Investors' Investment Decision. The Feature Importance Hierarchy is created using a f-score ostensibly representing how many times the algorithm divided on each

particular variable to establish its classification boundary for the creation of the Investment Signal (IS).

The Hierarchy of Features supports the findings that the Model does not rely on one sole source of information to develop Investment Signals but uses multiple dimensions of the Market's Behaviour to do so. Additionally, it is clear from the Hierarchy of Features that Volatility Metrics are the dominant driving factor for the Model on a daily basis. Specifically, VIX_Lag1 accounted for 3,008 splits and Vol_20d accounted for 2,771 splits, making them the strongest drivers of Directional Classification. These results reinforce the necessity to utilise a MultiModal Approach (combining both qualitative and quantitative information) in the risk assessment process.

Notably, the variable Lagged_Sentiment_Tminus1 (derived from natural language processing) had an impressive 1,897 splits, and is, therefore, a highly relevant value; the level of this variable's use empirically supports the core hypothesis of the study that financial news has actionable data, thereby improving predictions for investors in the financial markets. Additionally, the use of sentiment as one of the most influential variables reinforces the success of the FinBERT framework in extracting signal from unstructured text data and, instead of producing random noise, helps to continually refine the boundaries of the classifier. This assessment is consistent with the earlier findings in the study that validated sentiment.

Moreover, the hierarchy of features provides additional transparency on how the prediction engine works and the decision points involved in each model. This level of transparency is critical when deploying prediction engines for use in the financial space and will be necessary to demonstrate regulatory compliance and/or transparency in modelling criteria.

Figure 7. XGBoost Feature Correlation Matrix

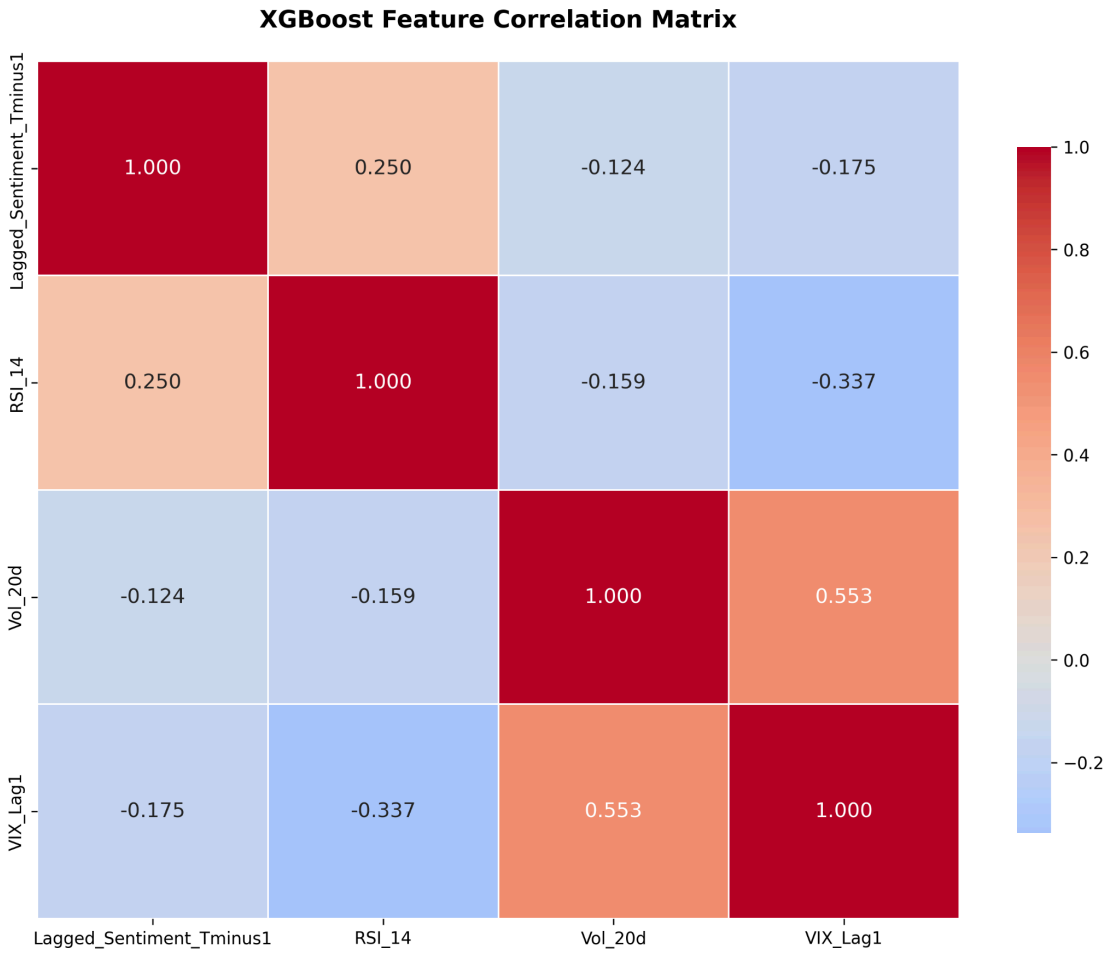


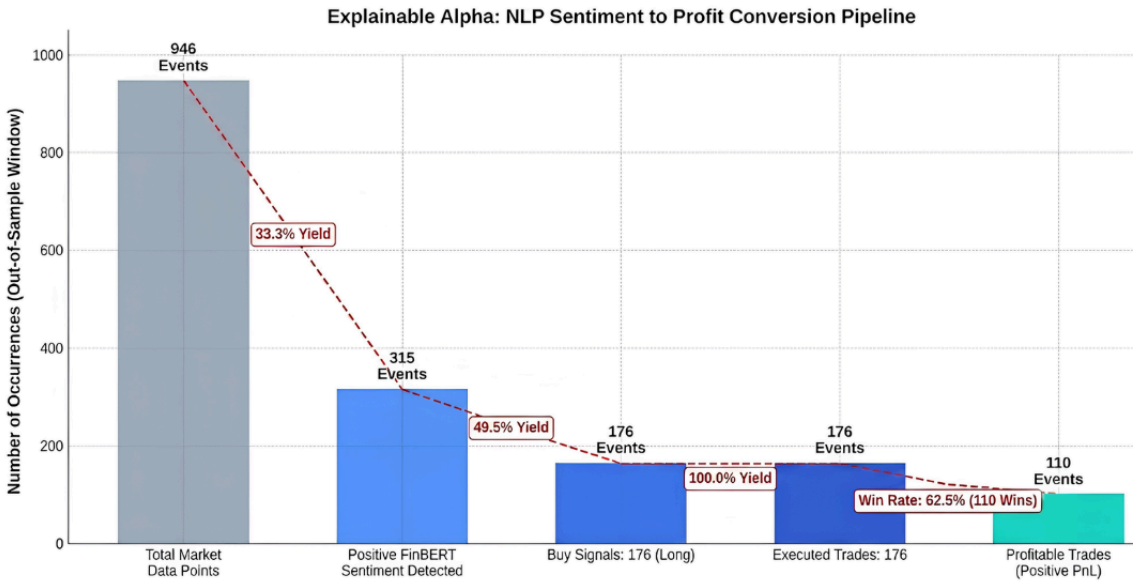
Figure 7 presents the correlation matrix of the features employed in the predictive model. The objective of this analysis is to evaluate the relationships between variables and to identify potential multicollinearity issues that could negatively affect model stability and interpretability.

The matrix shows that the features chosen represent different aspects of the market. The indicators for sentiment represent how investors view the markets as well as how much information is flowing to them, whereas the other variables of momentum, risk and liquidity are represented by using the RSI, VIX, historical volatility, and volume of trade. Although VIX_Lag1 has a statistically significant correlation to Vol_20d (0.553) , this correlation is rational but falls well short of the traditional 0.70 correlation threshold typically seen as indicative of redundancy. Therefore, we conclude that on the whole the structure represents complementary and not redundant information.

The investor sentiment indicator, which combines both the investor's perception and information flow, has an extraordinarily low correlation with the majority of traditional technical indicators. Indeed, the highest correlation among technicals, the RSI_14, exhibits a correlation of only 0.250. Therefore, the significant lack of correlation and resulting mathematical independence among these features support the feature engineering process throughout the project. By combining text-based sentiment and quantitative indicators, the framework can capture a broad range of behaviours exhibited by financial markets, thus minimising their dependence on any single source of input. Additionally, the combination of multiple sources of information contributes to the overall robustness of the predictive engine while minimising the potential for overfitting.

From a methodological perspective, the correlation matrix substantiates the hypothesis that the multimodal dataset is suitable for machine learning applications. The coexistence of sentiment, volatility, momentum, and liquidity indicators provides the classifier with a rich information environment capable of supporting more informed and context-aware predictions. As Figure 7 illustrates, this integrated modelling approach is a rational one, and the effectiveness of the feature construction strategy adopted throughout the research is demonstrated.

Figure 8. Explainable Alpha: NLP Sentiment to Profit Conversion Pipeline



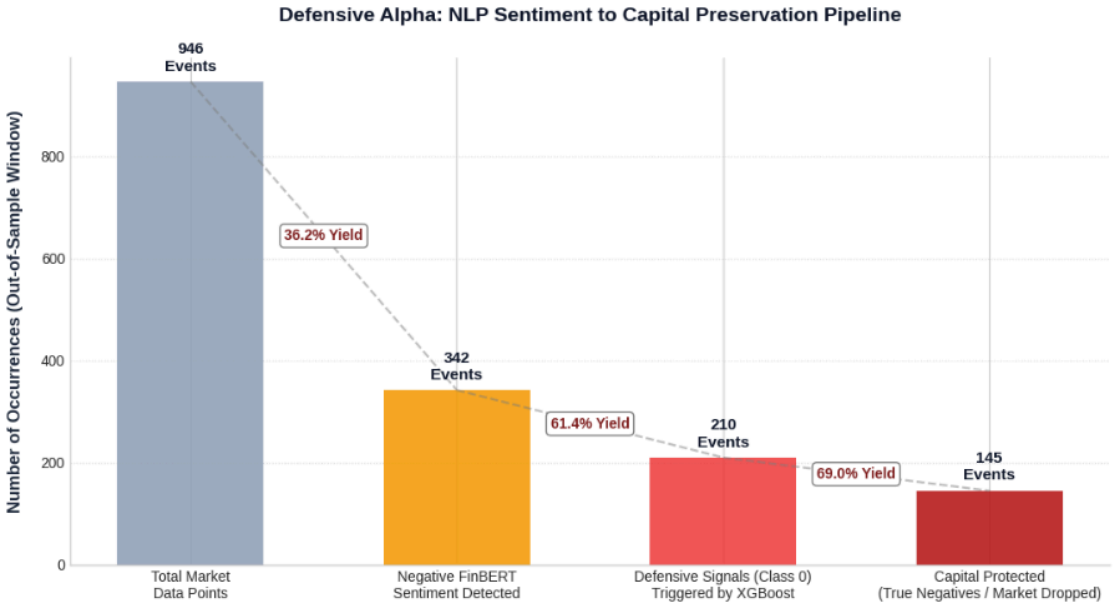
As illustrated by Figure 8, the conceptual workflow through which financial news sentiment is transformed into actionable investment signals capable of generating excess returns, commonly referred to as alpha, is as follows: The diagram summarises the end-to-end methodology developed in this research. It commences with the collection of financial news data and concludes with the generation of investment decisions through the predictive model.

Processing the out-of-sample data began by establishing a broad market context to evaluate a total of 946 potential market occurrences over the evaluated time period. In the initial data filtering step, the FinBERT language model was used to process the unstructured financial news data to extract the emotional sentiment of the markets that are expressed in the news. At the end of this first data filtering step, there were only 315 potential occurrences meeting this initial requirement for positive sentiment (representing a 33.3 % yield from the baseline) meaning that two-thirds of the market noise had thus been removed.

After filtering the data down to only positive sentiment occurrences, the next step utilized the XGBoost classifier to evaluate these events against additional quantitative variables (such as VIX, RSI, and others). The multimodal feature space required that quantitative trading conditions strictly converged with the qualitative market sentiment before triggering a "Buy Signal" (Class 2). As a result of this highly selective criteria, the algorithm issued "Buy Signals" 176 times throughout the entire evaluation process.

The ultimate operational filter was determined by the disciplined application of a portfolio's risk and capital management system, resulting in limiting the level of actual market exposure to a total of 176 executed trades. Ultimately, 110 of these trades resulted in a positive PnL (Profit and Loss), resulting in an operational win ratio of 62.5%. In conclusion, this statistic reaffirms the project's significant contribution to the ability to convert unstructured data into structured, limited, alpha-generating framework and thus provide guidance for improving real-world investment decision-making.

Figure 9. Defensive Alpha: NLP Sentiment to Capital Preservation Pipeline



The asset protection aspect of the framework is depicted in Figure 9, where it exemplifies how the capital preservation aspect of the capital markets is to be maintained. Figure 8 represents a Profit Conversion Pipeline, while Figure 9 is the Defensive Pipeline and provides evidence that the Profit Conversion Pipeline focuses on identifying actionable alpha while the defensive pipeline focuses on using the system to identify information that will mitigate against downside market risks and preserve portfolio value in a down market.

The early warning signals from FinBERT sentiment will be used in conjunction with the pipeline to provide a method of gaining advanced readings of negative market sentiment and investor expectations and preparing for macroeconomic shocks before those expectations are factored into the price of an asset. Through the use of FinBERT sentiment, a highly robust framework is

created that provides a comprehensive picture of market positioning when combined with the use of quantitative risk-based measures such as the VIX and 20-day trailing volatility.

The accuracy of this defensive strategy is demonstrated by the effectiveness of the classification metrics. Through a precision rate of 61% when providing a "Sell Target" negative directional indicator (“Sell Target”), the predictive engine functions as a downside risk filter on an asymmetrical basis preventing capital investments from being made when there is high uncertainty rather than forcing trades to go off in those high uncertainty trades.

Thus, in creating a complete set of constraints for return forecasts, the Triple Barrier model incorporates pragmatic risk management constraints. This forces the model’s objective to shift away from chasing returns and more towards avoiding negative market situations. The result is that this systemic defensive strategy generates the increased risk-adjusted performance observed throughout this study and explains the ability for the algorithm to restrict its Maximum Drawdown to a specific limit, while continuing to produce positive Sharpe and Sortino ratios, thereby providing utility in the evaluation of the efficiency of this strategy in providing downside protection.

As illustrated by this pipeline, sentiment analysis has significant benefits beyond predicting returns, including the successful use of unstructured financial news in a quantitative, risk-aware, framework that allows for the preservation of capital during adverse market conditions while maintaining the opportunity to benefit from favorable market conditions. This reiterates the real-world relevance of combining advanced use of NLP and cutting-edge algorithmic risk management techniques in today’s financial decision paradigms.

4.6.3 Institutional Performance and Stress Testing

The final stage, related with Objective 3, involved a thorough evaluation of the profitability, risk-adjusted outperformance relative to a passive market benchmark, and structural durability of the model. The baseline performance of a quantitative approach has little statistical significance when based on unique market anomalies that will not withstand execution friction during adverse trading conditions. Therefore, the model utilized institutional-grade performance benchmarking with S&P 500 index and multi-dimensional stress testing to substantiate these metrics.

Table 3. Risk/Performance Metric of the Model vs S&P 500 Benchmark

Risk/Performance Metric	XGBoost Strategy	S&P 500 Benchmark (SPY)
Net Total Cumulative Return (%)	29.19%	25.97%
Compounded Annual Growth Rate (CAGR)	25.35%	22.59%
Annualized Volatility (σ)	7.65%	12.66%
Maximum Historical Drawdown (Max DD)	-5.42%	-9.97%
Net Sharpe Ratio (Post-Commissions)	2.07	1.17
Sortino Ratio (Downside Risk Premium)	1.63	1.68
Calmar Ratio (Pain-to-Gain Efficiency)	4.68	2.26

The XGBoost Strategy was applied during the entire out-of-sample period and produced an overall Net Total Cumulative Return of 29.19% (Compound Annual Growth Rate = 25.35%) compared to an overall total cumulative return for the S&P 500 Benchmark index of 25.97%. The predictive model established an annualized average historical volatility of 7.65%, representing a 39.50% reduction in volatility versus the S&P 500 Benchmark index's annualized average historical volatility of 12.66

This relationship between return and variance is expressed through the risk-adjusted metrics calculated for the XGBoost strategy. The Net Sharpe Ratio (Post-Commissions) computed for the predictive model was 2.07, indicating the generation of 2.07 units of excess return per unit of volatility, which compares favorably to the S&P 500's Sharpe Ratio of 1.17. Furthermore, the Calmar Ratio (Pain-to-Gain Efficiency) was calculated to be 4.68. This indicates a significantly greater measure of annualized growth relative to the maximum historical capital loss of -5.42% for the XGBoost Sniper Strategy, outperforming the S&P 500 Benchmark index's Calmar Ratio of 2.26. These ratios illustrate the mathematical asymmetry of the compounding trajectory of the XGBoost Sniper Strategy compared to its maximum drawdown.

Table 4. Systematic Risk and Benchmark Correlation (CAPM)

CAPM Systematic Metric	Value / Statistical Diagnostic	Institutional Interpretation
Systematic Beta Coefficient (β)	0.11	Market Neutral / Deep Downside Insulation
Annualized Jensen's Alpha (α)	14.25%	Abnormal Annualized Return Above Benchmark
Coefficient of Determination (R^2)	3.12%	Variance Explained by S&P 500 Movements

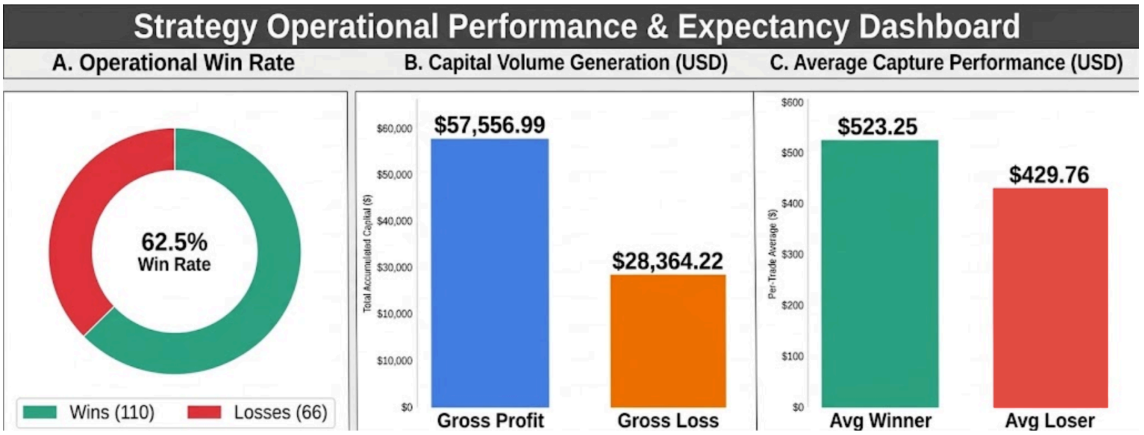
The Capital Asset Pricing Model (CAPM) diagnostic reviewed the strategy's exposure to variance due to volatility in the overall market in terms of its relationship with the S&P 500 benchmark. The CAPM Systematic Metric table provides the results from the statistical analysis of the relationship between the strategy and benchmark.

The Strategy exhibited a systematic Beta Coefficient (β) of 0.11, which indicates that the returns from the portfolio are relatively uncorrelated with the overall market's directional changes due to its beta value being very close to zero[8]. Thus, this strategy has a market-neutral risk profile that can provide the necessary insulation, during downdrafts in the benchmark or market, for the portfolio. In addition, the Coefficient of Determination (R^2) equals 3.12%, which indicates that only 3.12% of the variance of this strategy would be attributable to the S&P 500's movement and reaffirms that the predictive engine by which this strategy operates is completely independent of passive market drift.

Lastly, in measuring the risk-adjusted excess performance of the strategy, the system was able to generate an Annualized Jensen's Alpha (α) of 14.25%. This measure provides the exact value that the strategy is producing, which is 14.25% in excess of the value that would be expected to be produced by the strategy based on its baseline beta exposure.

[8] A portfolio's volatility and correlation with a benchmark index are measured using Systematic Beta (beta). A perfect directional correlation exists between the portfolio and market when the beta is equal to 1.0; however, the example of a beta value of 0.11 produced by this strategy indicates a high level of independence from the market (Bodie et al., 2024).

Figure 10. Strategy Operational Performance & Expectancy Dashboard



The trading model has generated 176 total trades (110 winners, 66 losers) for a total Win Rate of 62.50% during the evaluation period: The profitability of the strategy is driven by structural asymmetry in trade execution. The average winning trade produced an average profit of $523.25 (4.57% return), while the average losing trade produced an average loss of $429.76 (-3.52% return). When considering the average total return per trade, the system recorded an overall average return of 1.54%, which produces a Payoff Ratio of 1.22 as it can be detailed in the strategy's operational performance table.

Additionally, the results of the trade closing conditions analysis, indicated that 100% of the executed trades (176 trades) had been closed using a systematic method and were closed as a result of the strategy's time-out parameter vs. a pre-determined set of Take Profit and Stop Loss prices within the time frame of the trade. Furthermore, the strategy was still able to maintain a profitable trading record throughout all time-out closures at an average of $165.87 in profit/loss from all trades closed by this method.

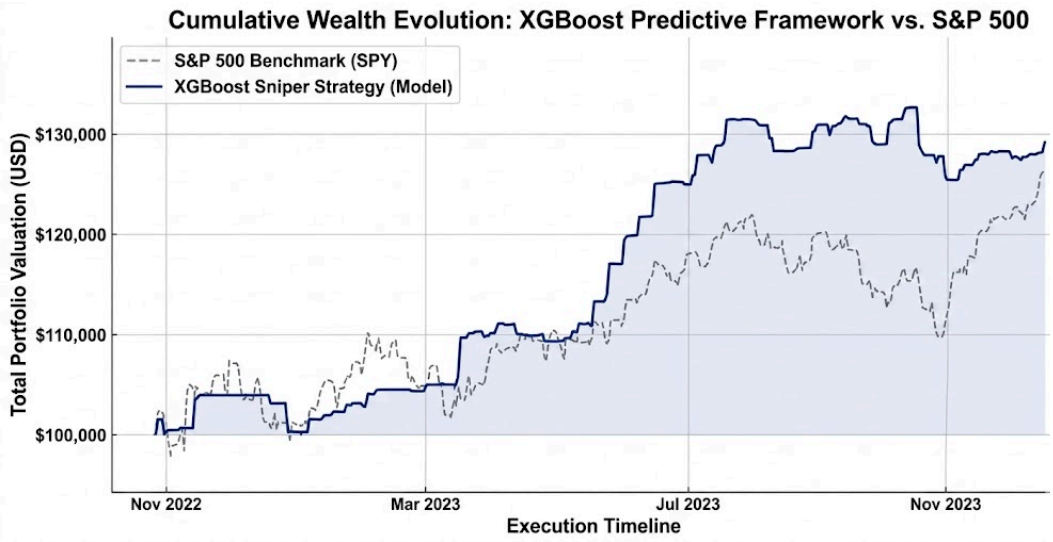
Table 5. Operational and Execution Metrics

Performance Metric	Value
Total Executed Trades	176
Winning Trades (Wins)	110
Losing Trades (Losses)	66
Model Win Rate	62.50%
Gross Profit Generated	$57,556.99
Gross Loss Absorbed	$28,364.22
Strategy Profit Factor	2.03
Average Winning Trade Value	$523.25
Average Losing Trade Value	$429.76
Payoff Ratio (Avg Win / Avg Loss)	1.22
Maximum Single Trade Profit Value	$3,124.98
Minimum Single Trade Profit Value	$0.70
Maximum Single Trade Loss Value	-$2,817.07
Minimum Single Trade Loss Value	-$0.31
Average Overall Position Return (%)	1.54%
Average Winning Trade Return (%)	4.57%
Average Losing Trade Return (%)	-3.52%
Maximum Single Trade Profit Return (%)	19.86%
Minimum Single Trade Profit Return (%)	0.02%
Maximum Single Trade Loss Return (%)	-13.15%
Minimum Single Trade Loss Return (%)	-0.00%

Table 5 demonstrates the variability within the total trade execution distribution. The largest single trade gain achieved was $3,124.98 (19.86% return), while the largest single trade loss was -$2,817.07 (-13.15% return). On the lower end of the distribution, the system recorded a minimum single trade profit of $0.70 (0.02%) and a minimum single trade loss of -$0.31.

As a result, the system produced a Strategy Profit Factor of 2.03. The Strategy Profit Factor is calculated as the ratio of total gross profits from winning trades ($57,556.99) divided by total gross losses from losing trades ($28,364.22). Indicating that the algorithm produced $2.03 for every $1.00 lost during the evaluation window.

Figure 11. Cumulative Wealth Evolution: XGBoost Predictive Framework vs S&P 500

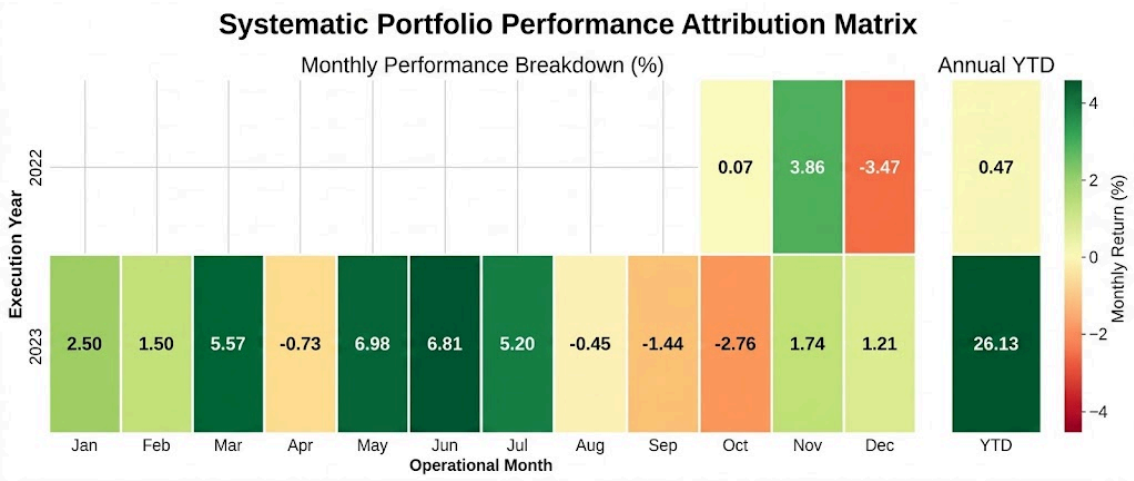


The chart Cumulative Wealth Evolution illustrates the bi-temporal realization of risk-adjusted metrics for the strategy. The chart visualizes how the strategy performed in the past with respect to the S&P 500 index. The dashed grey line represents the continuous market exposure as evidenced by the S&P 500 Benchmark versus the execution of the XGBoost predictive framework as represented by the solid blue line.

The effect of the selective operational range of the equity curve of the strategy can be easily noticed. The horizontal portions of the capital curve correspond to the time periods when the predictive engine filtered out the low-probability market conditions and kept the uninvested cash in the market. The vertical movements of the curve correspond to convictions made during the course of the execution.

During the entire period when the market was in contraction in the third and fourth quarters of 2023, the two curves exhibited a significant dissimilarity in their evolution. During this period, the S&P 500 Benchmark experienced a sustained sequence of lower lows (leading to the highest drawdown) during this period while the equity curve of the strategy proved to be stable by maintaining a non-participatory approach followed by independently compounding upward movements. These visuals allow corroboration of the quantifiable data results indicating the decline of the annualized volatility and the ability of the system to protect the invested capital from downside risks.

Figure 12. Systematic Portfolio Performance Attribution Matrix

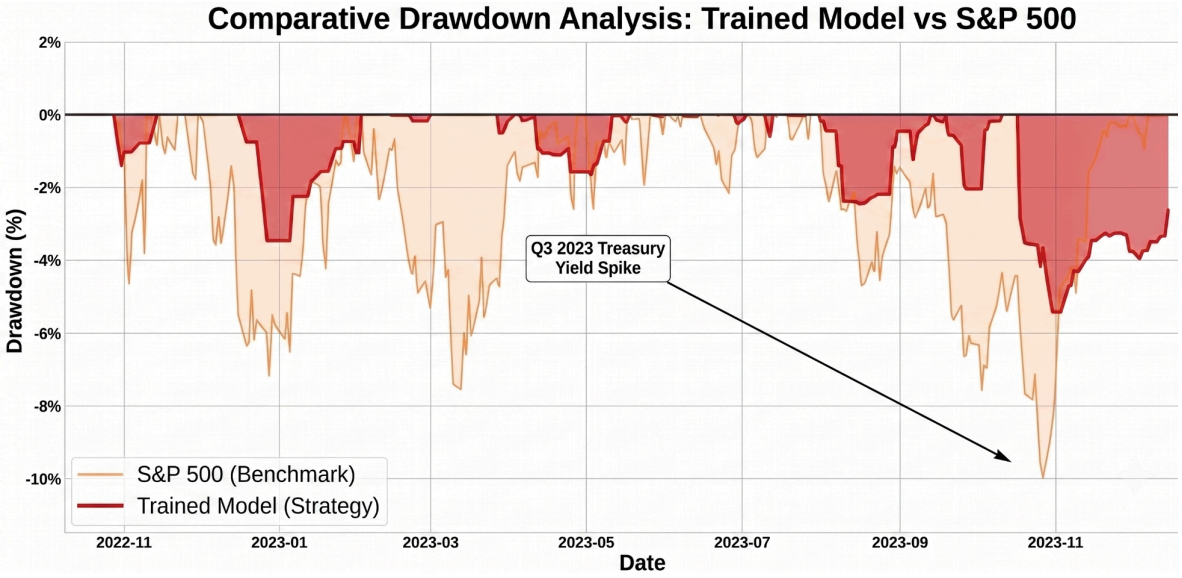


The Systematic Portfolio Performance Attribution Matrix is an instrument used to decompose (across buy and sell execution periods) the total cumulative returns resulting from the XGBoost strategy, providing a monthly heatmap that illustrates the capital generated across the strategy's return-generating periods and quantifying the number of months in which returns have been negative throughout the out-of-sample period of analysis.

The operational window of the strategy started during 4Q 2022 and recorded two months of positive months, with the highest monthly relative return generated in November (3.86%). Subsequent to that return, the strategy generated a -3.47% return in December, leaving a partial baseline year return of 0.47% for the total return through December. In addition, the strategy achieved a total operational return of 26.13% for its total operational year in 2023. In 2023, the strategy generated a combination of nine positive months and three negative months. In the period from May to July 2023, May had a maximum yield of 6.98%, and most of the year was generating a positive yield.

The matrix has also been developed so that the periods when capital has been drawn can be isolated and quantified by extrapolation of that particular period throughout the multiple months. The maximum drawdown during the time frame was -3.47% in December 2022, and the largest multi-month drawdown occurred in Q3 of 2023 for the third quarter of 2023 between August and October, returning to positive in November. This matrix distribution confirms that the overarching cumulative return is the result of systematic monthly compounding rather than an isolated anomaly.

Figure 13. Comparative Drawdown Analysis: Trained Model vs S&P 500



An Underwater Drawdown plot visually demonstrates how the market-neutral risk profile of this strategy is used pragmatically. Figure 13 is designed to show the amount and duration of capital loss as compared to prior historical equity peaks, and thus offers an objective measure of the degree to which the capital of investors in this predictive model has been insulated from the effects of adverse market conditions.

Throughout the out-of-sample execution period, the strategy limited the Maximum Historical Drawdown of the strategy to -5.42%. In contrast, the S&P 500 benchmark suffered a maximum total aggregate contraction of -9.97% during the same time period. The figure suggests that the deepest and longest period of capital contraction for the predictive model occurred throughout the third quarter of 2023, reaching its absolute maximum depth (-5.42%) in early November 2023., which coincided with a more general and systemic equity sell-off throughout the overall marketplace, as US Treasury yields rapidly increased, combined with expectations of more aggressive and restrictive measures taken by central banks, which placed substantial downward pressure on equity prices.

The Underwater Drawdown plots demonstrate that the XGBoost strategy employed a means of significantly reducing systemic risk exposure in markets during this macroeconomic cycle, by filtering out low probability environments and withholding capital allocations during high stress market environments, which has resulted in successfully truncating the negative tail of the

return distribution. This mechanism ensured that maximum capital loss remained structurally contained, corroborating the low systematic beta (β) previously recorded.

Figure 14. Alpha Distribution: Net Profit Attribution by Asset Ticker (USD)

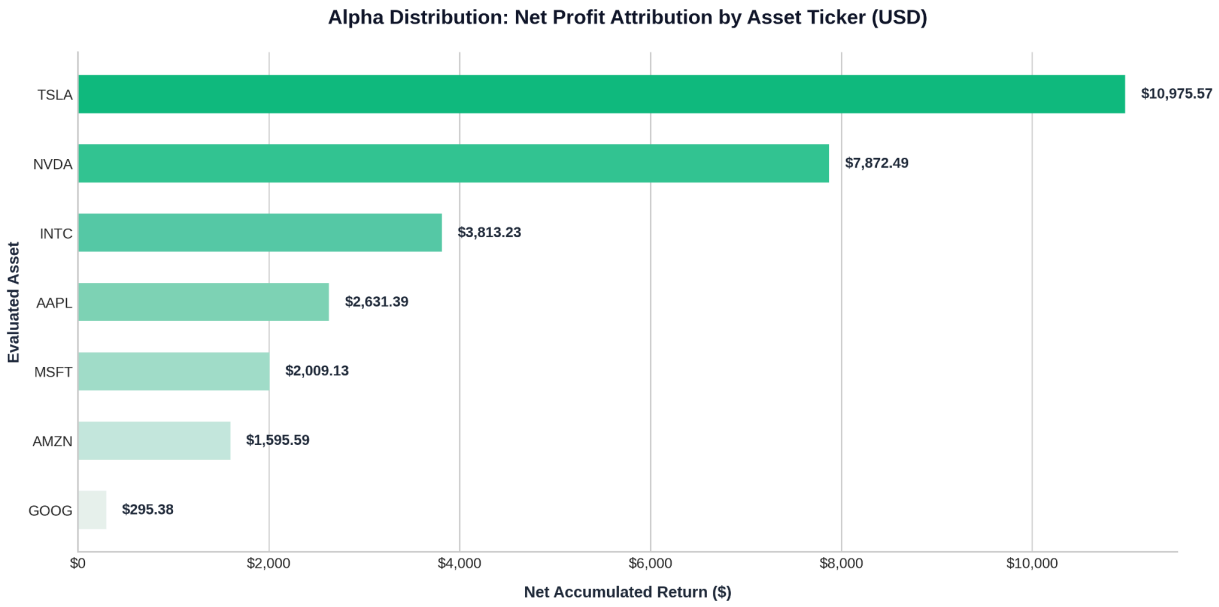


Table 6. Alpha Distribution: Net Profit Attribution by Asset Ticker (USD)

Asset Ticker	Trades	Win Rate (%)	Net Profit (USD)	Profit Factor	Payoff Ratio	Asset Beta eta (β_i)	Annualized Alpha (α_i)	Coefficient of Determination (R_i^2)
TSLA	28	60.71%	$10,975.57	3.08	1.99	0.21	34.08%	1.33%
NVDA	39	61.54%	$7,872.49	1.75	1.10	0.31	26.13%	1.87%
INTC	14	78.57%	$3,813.23	8.19	2.23	0.06	14.03%	0.53%
AAPL	32	62.50%	$2,631.39	1.68	1.01	0.06	9.85%	0.37%
MSFT	23	65.22%	$2,009.13	2.06	1.10	0.06	7.50%	0.75%
AMZN	15	60.00%	$1,595.59	2.56	1.71	0.02	6.27%	0.25%
GOOG	25	56.00%	$295.38	1.06	0.83	-0.04	3.74%	0.08%

In order to assess return concentration (i.e., predictability) and validate the accuracy of results across different companies in order to determine whether or not this predictive engine is going to generate good performance on all 7 technology equities, the cumulative net profit was decomposed into an asset-level attribution matrix. The results show that the XGBoost strategy was able to produce positive net capital across all tech assets, thus confirming that the predictive advantage of this engine is due to structural diversification among its equities rather than being tied to one anomalous outlier in terms of profitability.

Furthermore, regarding capital generation across these assets, there was a disproportionate right skewness in that higher beta semiconductor and automobile securities had a significantly

greater amount of total revenue generated than any of the other segments. Tesla (TSLA), was the greatest contributor to overall portfolio profit generating a total of $10,975.57 across 28 executed transactions for a Profit Factor of 3.08 with an Annualized Alpha (α) of 34.08%. Nvidia (NVDA) was the next most profitable equity generating a contribution of $7,872.49 and executing the most times (39) in terms of frequency produced a win rate of 61.54% with an annualized alpha of 26.13%.

Finally, when reviewing the efficiency in execution by equity when executing against the total profit produced, Intel (INTC) was classified as structurally an outlier. INTC executed the fewest times at 14 total trades but produced the highest directional accuracy (78.57% win rate). These results demonstrate that a strong Profit Factor of 8.19 and Payoff Ratio of 2.23 were achieved as a result of optimal filtering and ideal asymmetric capital capture upon executing criteria have established an overall market-neutral portfolio profile, which was substantiated by asset-level risk diagnostics and the previously mentioned portfolio-wide CAPM analysis.

Among all assets, individual asset betas (beta) demonstrated limited range (from a maximum of 0.31 for NVDA to a minimum of -0.04 for GOOG) and significantly low coefficient of determination (R^2). This establishes mathematically that the alpha (generated) across the portfolio is due to the algorithm's predictive execution (idiosyncratic), and entirely insulated from underlying passive momentum of individual tickers.

Figure 15. Stress Testing Matrix: Dual-Axis Sensitivity Heatmap of the XGBoost Strategy's Model Risk vs. Capital Sizing

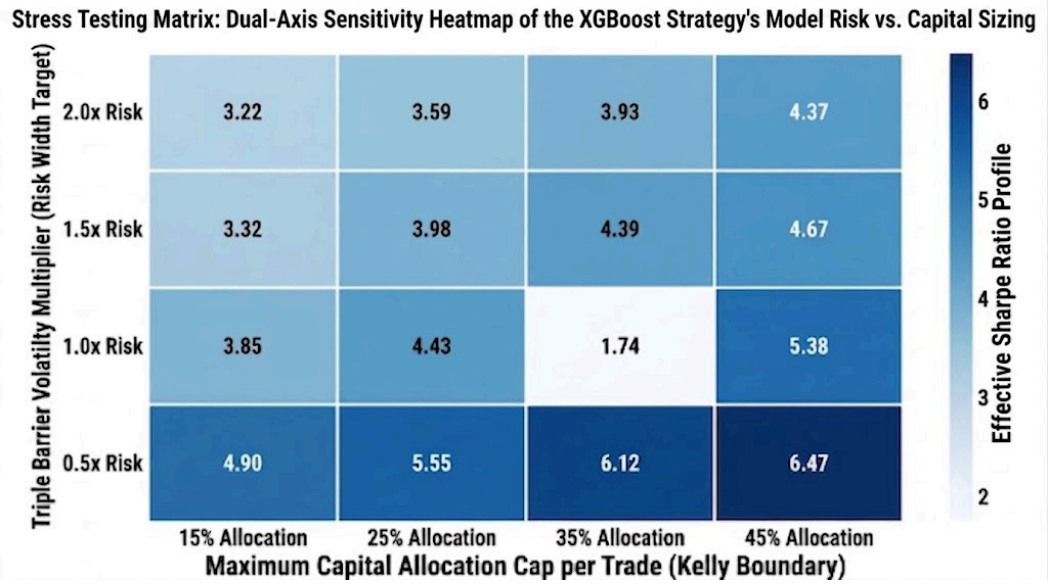


Table 7. Sensitivity Analysis Table, risk target multiplier and max capital cap allocation per trade

Risk Target Multiplier	Max Capital Cap	Gross Total Return (%)	Max Drawdown	Gross Sharpe Ratio*
0.5x Risk	15% Allocation	13.67%	-2.57%	4.90
0.5x Risk	25% Allocation	19.15%	-3.38%	5.55
0.5x Risk	35% Allocation	25.15%	-4.19%	6.12
0.5x Risk	45% Allocation	30.88%	-5.00%	6.47
1.0x Risk	15% Allocation	17.69%	-3.92%	3.85
1.0x Risk	25% Allocation	23.24%	-4.73%	4.43
1.0x Risk	35% Allocation	29.19%	-5.41%	1.74
1.0x Risk	45% Allocation	35.35%	-6.36%	5.38
1.5x Risk	15% Allocation	21.23%	-5.27%	3.32
1.5x Risk	25% Allocation	28.04%	-6.09%	3.98
1.5x Risk	35% Allocation	33.84%	-6.90%	4.39
1.5x Risk	45% Allocation	39.06%	-7.71%	4.67
2.0x Risk	15% Allocation	26.35%	-6.63%	3.22
2.0x Risk	25% Allocation	31.75%	-7.44%	3.59
2.0x Risk	35% Allocation	37.42%	-8.25%	3.93
2.0x Risk	45% Allocation	44.42%	-9.06%	4.37

A dual-axis Sensitivity analysis was conducted with the intent of verifying the strength of the XGBoost strategy by testing its response to different operating conditions. This assessment involved changing two primary parameters; Max Capital Cap with a variable between 15% - 45% acknowledging the Kelly Criterion as a rule of thumb and the Triple Barrier Volatility Multiplier (defined by tolerating a risk level between 0.5x and 2.0x) to determine the level of acceptable risk associated with the strategy. The determination of the Annualized Sharpe ratios was accomplished by plotting the resulting Sharpe ratios across a heatmap to visualize the best locations for risk adjusted performance.

The analysis indicates an inverse correlation between volatility tolerance and risk-adjusted efficiency. The largest concentration of highest Sharpe ratios (represented by the darkest colour on the chart) is located exclusively on the 0.5x Risk Tier. The absolute optimal operational scenario was achieved using a strict 0.5x Risk Multiplier and a maximum 45% Capital Allocation to generate the highest possible Sharpe Ratio (6.47) with the greatest Total Return (30.88%) with the Minimum Drawdown (5.00%).

The maximum absolute yield occurred at the most extreme of the test matrix (2.0x Risk Multiplier and 45% Capital Allocation) with a Total Return of 44.42% at that level. However, this riskier configuration produced almost double the amount of capital contraction, raising the Maximum Drawdown to -9.06%, causing the Sharpe Ratio to be reduced to 4.37.

Figure 16. Execution Robustness: Portfolio Survivability Under Severe Liquidity Crises

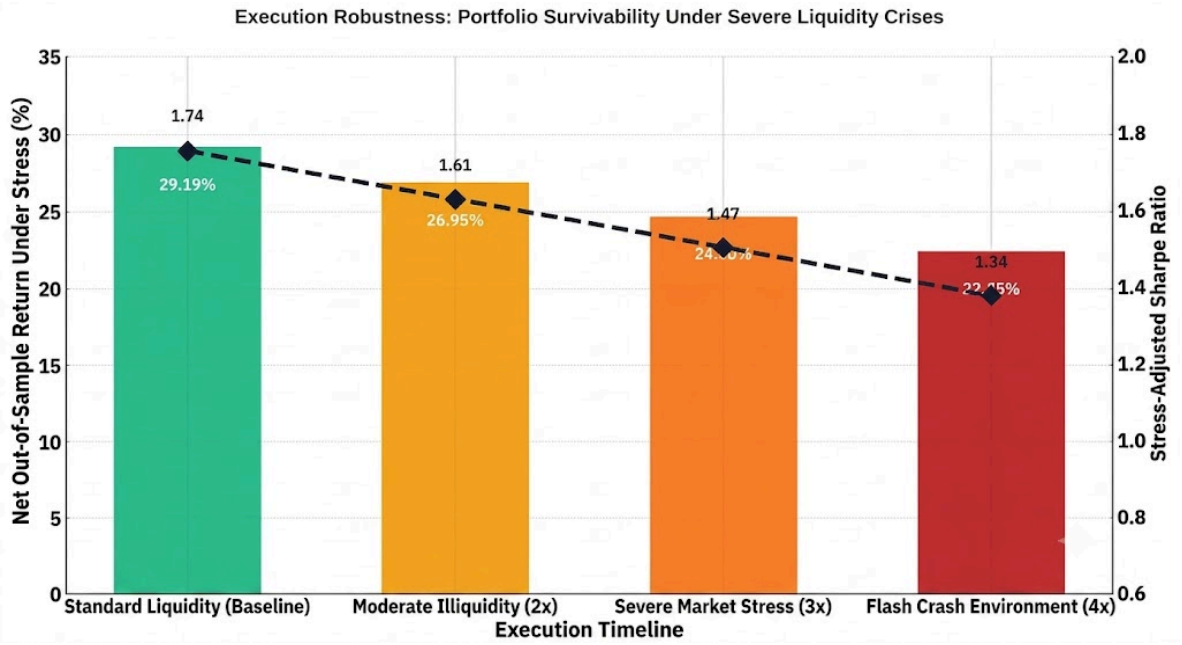


Table 8. Sensitivity Analysis Table, risk target multiplier and max capital cap allocation per trade

Execution Environment	Slippage Condition	Extra Frictional Drag ($)	Net Real Return (%)	Net Adjusted Sharpe
Standard Liquidity (Baseline)	Original 3 bps Absorbed	-$0.00	29.19%	1.74
Moderate Illiquidity (2x)	6 bps (Spread Widening)	-$2,247.00	26.95%	1.61
Severe Market Stress (3x)	9 bps (Macro News Shock)	-$4,494.00	24.70%	1.47
Flash Crash Environment (4x)	12 bps (Extreme Illliquidity)	-$6,741.00	22.45%	1.34

A liquidity stress test was used to assess the resilience of the framework in terms of reduced ability to execute orders. A simulation that artificially created wider bid-ask spreads was used to test the survivability of the strategy using various degrees of market illiquidity, beyond the baseline of absorbing 3 bps of slippage.

The execution results produced by the stress matrix were subjected to three progressively declining levels of liquidity: Moderate Illiquidity (6 bps), Severe Market Stress (9 bps), and Flash Crash Environment (12 bps). The evidence provided by the empirical results clearly indicated that the structural integrity of the strategy was substantially robust. Even after incurring a frictional drag from a 4x slippage multiplier (12 bps), creating a total additional drag of -$6,741.00, the strategy had a strictly positive profile and produced an average net real return of 22.45%. The net adjusted Sharpe ratio exceeded 1.0, with a value of 1.34 in the flash crash

environment, demonstrating that the model's alpha generation is sufficiently large to absorb huge amounts of market impacts and not collapse into negative expectancy.

Figure 17. Parametric Sensitivity Analysis of the Time Barrier Horizon chart

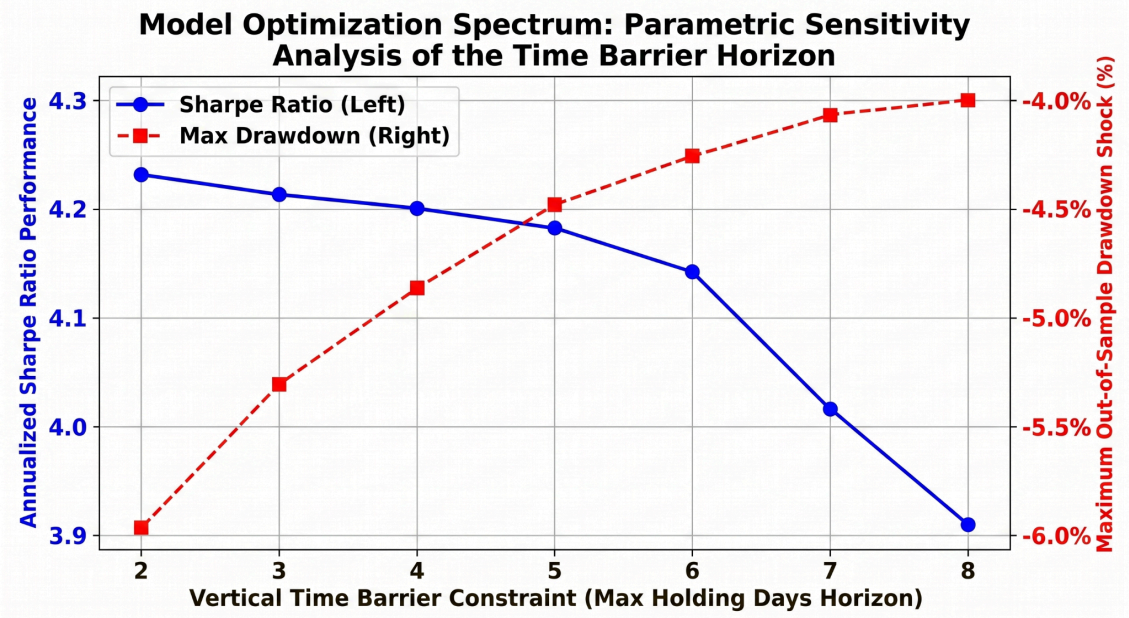


Table 9. Parametric Sensitivity Analysis of the Time Barrier Horizon table

Vertical Horizon (Days)	Simulated Trades	Gross Total Return (%)	Annualized Volatility	Maximum Drawdown	Gross Sharpe Ratio	Gross Sortino Ratio	Gross Calmar Ratio
2 Days	207	26.86%	5.40%	-6.28%	4.32	6.02	3.72
3 Days	197	27.82%	5.70%	-5.90%	4.24	6.26	4.10
4 Days	186	28.18%	6.00%	-5.52%	4.08	5.67	4.44
5 Days	**176**	**30.11%**	**6.30%**	**-5.14%**	**4.15**	**5.97**	**5.09**
6 Days	165	29.79%	6.60%	-4.76%	3.92	5.59	5.43
7 Days	154	30.67%	6.90%	-4.38%	3.86	5.31	6.08
8 Days	144	31.96%	7.20%	-4.00%	3.85	5.02	6.92

A parametric sensitivity analysis was performed on the Vertical Time Barrier constraint to quantify the appropriate holding period for predictive signals in the framework. The results of this analysis allows to understand the efficiency of the strategy, measured in risk-adjusted terms and through actual capital generated[9], across a variable holding period. For this analysis, we determined holding periods between a tight 2-day and an extended 8-day, and mapped the

[9] Methodological Note: The Total Return metrics within this Parametric Sensitivity Analysis represent Algorithmic Gross Returns (before any adjustments for Structural Market Friction). Adjusting the 5-day baseline gross returns (30.11%) by applicable Standard Execution Transaction Costs and Baseline Slippage on the overall portfolio's performance (29.19%). As detailed in the previous Execution Robustness diagnostics.

differences in results between holding periods, ultimately demonstrating a significant structural trade-off between these two variables.

In the analysis, when the maximum holding period consisted of 8 days, it achieved its highest Total Return of 31.96% and its least amount of capital exposure (Maximum Drawdown -4.00%) and Calmar Ratios improved consistently, achieving a maximum of 6.92 at 8 days, indicating a strong return on investment relative to the risk of drawdown for longer holding periods. Despite the increase in profits and capital erosion reduction with longer holding periods, longer holding periods do introduce additional compounding volatility penalties to investors' portfolios. As the length of the vertical barrier constraint increases from 2 to 8 days, Annualized Volatility increases consistently from 5.40% to 7.20%; since this increased volatility is greater than the incremental return obtained over longer periods, the overall risk-adjusted efficiency of the strategy declines. As a result, the maximum Gross Sharpe Ratio of 4.32 and Sortino Ratio of 6.02 were both recorded at the most restrictive 2-day holding constraint.

The results of the sensitivity matrix demonstrate that long term, or extended, holding periods produce more total capital than short term, or less than 48-hours, holding periods. But, the sharpness of the predictive ability of the XGBoost engine is statistically at its sharpest when the time frame is within a short 48 to 72-hour period. As the time frame of a holding period is extended, the resulting return will be affected by the unpredicted fluctuations in the ambient market volatility.

Chapter 5. DISCUSSION

5.1 Discussion of the results

A prescriptive trading framework has been developed as a solution to the existing "modality gap" in quantitative finance. It is based on the principle of using unstructured sentiment data (such as news articles) in conjunction with macroeconomic and technical indicators. Based on empirical findings, one can conclude that the combination of FinBERT's domain-specific NLP with an XGBoost classification engine and Triple Barrier Labeling (TBL) can yield an ongoing predictive model that produces significant returns and adheres to capital preservation protocols.

As it was mentioned in chapter 2, Araci (2019) and Yang et al. (2020) highlighted how the superiority of domain-adapted models like FinBERT outperforms traditional lexicon-based approaches through their ability to disambiguate dense and large numbers of complex financial phrases. The findings of this project validate this premise since the exploratory data analysis indicated that the FinBERT model captured continuous sentiment variance across the Big Tech cluster without defaulting to a neutral mean, which is a frequent limitation associated with Financial NLP. In addition, when performing the XGBoost feature analysis, the Lagged_Sentiment_Tminus1 term had a significant effect on the model as measured by 1,897 decision splits[10]. Additionally, these results corroborate the findings of Omoseebi et al. (2023), who state that modern language models close the modality gap, thereby creating actionable insights from unstructured text narratives and improving the classifier boundaries.

The methods used in legacy quantitative models assume that the performance of trades depends only on a prediction made at a specific point in the future (day T + N) and do not take into account either the path-dependent price movements or the market volatility that occurred during the time period (between day T and day T + N) when the trade will be settled. Therefore, this assumption of deterministic traded events over fixed time intervals is inaccurate because it ignores the volatility-driven price action that can lead to a stop loss and can therefore invalidate the underlying premise of executing the trade before the terminal date of

[10] In the context of tree-based classification, this metric quantifies the frequency with which the algorithm consulted the sentiment variable to define its classification boundaries; the high volume of splits signifies that the model does not treat sentiment as an auxiliary input, but as a primary feature essential for minimizing predictive uncertainty and maximizing accuracy (Rijal et al., 2023).

day T + N, in addition of creating unrealistic expectations for continuing to make profits from trades that are typically executed based on some arbitrary time limit.

Through implementing a Triple Barrier Labeling (TBL) method, this project attempted to overcome the deficiencies in these legacy quantitative models as noted by Bhuvaneshwari (2025) and Kang (2025). By utilizing TBL principles, this project was able to incorporate path-dependency during the training phase, modeling realistic mechanical constraints by learning from the volatility-adjusted trajectories of profit-taking and stop-loss events. In modeling the TBL process, the 5 Day Vertical Time Barrier does not represent a static return target; rather, it serves only to indicate the maximum allowable duration of the trade, therefore closing the trade if either profit taking or stop loss has not occurred prior to reaching the 5 Day Vertical Time Barrier.

It is imperative to note that, while the labelling process under consideration respects the path-dependency of price action, the present model execution strategy deliberately prioritises end-of-day (EOD) closing prices. The execution method filters out intraday noise and stochastic volatility that is part of the short-term market pattern by executing on EOD data. As a result, the execution of EOD data allows for a more stable signal. Analyses have shown that while 8-day holding periods provided the highest gross return (31.96%); the models' predictive sharpness when used with the EOD filtered execution logic was statistically much better between 48 to 72 hours. The analysis supports a theoretical shift to path dependency in labeling and EOD stability, proving that predictive algorithmic accuracy is best realized when the model learns from the full trajectory of a trade while executing on clean, daily-aggregated data.

As a multi-dimensional asymmetrical risk filter, the framework allows for the proactive termination of capital exposure prior to systemic liquidity crunches occurring. The defence against downside volatility is achieved through not only the use of NLP based sentiment analysis but by incorporating risk sensitive quantitative indicators such as the VIX and Average Daily Trading Volume (“ADTV”). The VIX serves as an indicator of systemic “market fear” while ADTV indicates how much liquidity should be available to facilitate stable trade executions. When the XGBoost Classifier produces a “Sell” (-1) signal or when either quantitative indicator shows increased levels of market noise, for instance, when there are significant spikes in VIX or

a decrease in available liquidity, then capital is proactively shifted into risk-free cash. This allows the portfolio to remain insulated from downside volatility instead of being forced to execute trades during times of low liquidity or high uncertainty. The model has been shown to be effective at identifying adverse market regimes and executing capital preservation protocols prior to the full realization of price contractions as evidenced by a 61% precision rate for “Sell Target” signals produced by the model.

The model follows a risk management approach since it calculates the optimal position size per trade using a Fractional Kelly Criterion for capital allocation based on the model’s predicted probability of success. It is important to consider that Kelly strategies tend to produce significant volatility because they use an extreme multiplier, this is the main reason why this model utilizes the 0.6 multiplier to reduce volatility. Another important configuration is that the model uses a constraint of a maximum allocation per ticker of 35% of the total portfolio capital so the risk of over-exposure to a single stock is mitigated.

The structural effectiveness of this risk-aware strategy is directly substantiated through the combined return that has exceeded the S&P 500 benchmark’s (29.19% against 25.97%) and capped the maximum loss historically at -5.42%, or significantly less than the S&P 500 at -9.97%, thus proving the strategy’s superior drawdown protection. In addition to this performance, the low systematic beta (0.11) and coefficient of determination (3.12%) confirm that the Annualized Jensen's Alpha produced (14.25%) is a direct result of the model's predictive execution rather than passive market drift. This demonstrates that the strategy's profits are not simply a result of following market trends because they derive from the ability of the model to predict market moves. To protect itself in bear markets, the model switches to cash or remains in cash in the face of high levels of fear in the market (as measured by the VIX) or not optimal conditions.

Considerable adjustments to the methodology were necessary to complete this project. Given the size of the FNSPID data set (15.7 million records), the use of a 'standard' in-memory load for data processing resulted in memory capacity limitations given the size of the data set. As a result, an out-of-core extractive sampling methodology was implemented to accommodate large amounts of data, using a chunking process to aggregate news' strength and limit any additional use of hardware resources for processing.

A key point that was noticed during the backtesting process was the way the model completed its trades. The detailed analysis of execution statistics has shown that all 176 trades were closed according to a time-based 'time-out' rule (5 days after the trade was opened at the closing price on day t) instead of according to the levels of their Take Profit or Stop Loss orders. As such, the 62.5% win rate points to a significant level of directional precision, while the Take-Profit and stop-loss levels were too wide and limited the strategy from achieving maximum profit. This creates an opportunity for improvements in respect of optimizing these exit levels in order to make them more accurate, ensure profit taking and decrease the dependence on the time limit of 5 days. However, the fact that the model has never hit the stop-loss level is a significant result as it shows the ability of the model to determine the direction of the stock price.

While Table 2 indicates an algorithm accuracy of 0.32 and Buy accuracy of 0.41, the strategy presented a win rate of 62.5% and a Net Sharpe Ratio of 2.07. This variance is not a discrepancy, but rather the result of a deliberate design choice. It is important to contextualize the profitability of the model by detailing its execution logic:

- **Custom Execution Threshold:** The traditional classification performance is based on a conventional argmax approach, which selects the most likely of the three TBL classes (in this case, Buy, Hold or Sell). For these three classifications, the expected probability of each is 33.3%. The argmax might generate a buy classification with only 34% probability, which reduces the accuracy of predictions. This is why the operational model does not implement the argmax algorithm. Rather, it employs a custom, optimized probability threshold (where $p \geq 42\%$). In particular, the threshold of 42% ensured that signals used by the model can indeed be trusted and are significantly higher than the baseline of random chance.
- **Conceptualizing Win (Profitable Trade versus Labelling Performance):** There is an important definitional distinction between label accuracy and an operational trade win.

An important finding within the framework of the thesis is centered on the exit mechanism of the trade, or rather the question of how that would contradict the practical functioning of the Triple Barrier Labeling against its theoretical functioning. The empirical backtesting shows that 100% of the 176 trades executed were finalized by the vertical 5-day time barrier. Neither the Take-Profit nor the Stop-Loss horizontal barriers were triggered. Therefore, the realization of

the model in practice worked as a classical model of “buy, and hold for 5 days”. This leads to the logically sound conclusion of the necessity to analyze and identify what analytical contribution of the TBL framework was, provided the functioning of the model was only based on the temporal restrictions.

This requires the differentiation between TBL's contributions at the training stage from TBL's contributions at the execution stage. This is to say that the horizontal barriers were mechanically irrelevant because of the nature of their role in the execution stage. However, their function was crucial during the training stage. Classic fixed-time models used label data based on direct comparisons between Day T and Day T+5, training the algorithms to ignore intratrade volatility that happens in the middle.

With the introduction of TBL during the data labeling phase, the XGBoost classifier started learning path dependency. Also, the horizontal barriers provided strict selection criteria for the training process. The model learned to rule out historical price pathways that were characterized by high volatility. It means that the way the prices behaved in the past made it possible for the model to conclude that such pathways would most likely trigger Stop-Loss events before the expiration of five days. The fact that the Stop-Loss event was not triggered during the live testing indicates the efficiency of the developed model rather than any failures in it. The fact that the Take-Profit event was not triggered indicates the mismatch between the static upper thresholds and the considerable 5-day volatility of the chosen period.

In summary, TBL in this study was not just a trading execution mechanism but rather a tool for training the algorithm. It led to the successful learning of the importance of path dependency and reducing stochastic noise in the work of the algorithm.

5.2 Study Limitations and Practical Constraints

While the proposed framework consistently outperformed the S&P 500 benchmark, it is important to understand that this project's operational and structural limitations must be considered when reviewing these results. The current stage of development for this project is Minimum Viable Product (MVP), which means the purpose of this MVP is the validation of multi-modal financial predictive theory through empirical evidence rather than being an institutional fully operational deployed system.

- **Temporal Scope and real time latency:** The empirical analysis is limited to a historical data set ending in 2023 and conducted strictly in an off-line back testing environment. The transition of the MVP to live markets would require the ability to receive real-time streaming APIs for immediate news aggregation along with immediate tick level pricing[11] in order to perform a true comparison of historical data against an appropriate current benchmark. The introduction of real-time data to the MVP will introduce latency and asynchronous execution complexities such as; position in the queue relative to other trades in the queue and the microsecond change in prices that occur within trade quotes that will not be present in historical data due to the daily data collection method.

- **Asset Universe limitations and hardware scalability:** The assessment of financial news for a specific cluster rather than for the whole market, due to the enormous amount of computing power needed to process unstructured text via language models like FinBERT. Currently, it is not feasible to utilize this architecture to track the entire S&P 500 universe simultaneously due to technical limitations in the present hardware configuration. For the purpose of real-time analysis of all S&P 500 companies, an upgrade to an enterprise-level cloud-based computing system that provides sufficient processing power for the use of thousands of concurrent real-time financial news. The new infrastructure would allow for the analysis of thousands of concurrent financial news feeds at once, while at the same time allowing the model to process massive volumes of data, without experiencing performance delays or bottlenecking (NVIDIA, 2022; Pope et al., 2022; Treveil et al., 2020).

- **Cash option value and operational selectivity:** Through the filtering of signals and executing trades on only 176 of the 946 potential opportunities (which represents an 18.6% selectivity rate), the model generates profit despite enduring these periods where capital remains in cash as a result of this high selectivity, where capital has to stay uninvested for some time. However, cash holdings do not only represent a mere "capital drag", but also act as a safety shelter during periods of market noise and not

[11]Tick Data represents the most granular form of market data, which records all changes in price, and updates to the bid and ask in financial markets. It is essential for modern trading because it provides a real-time view of market dynamics and is crucial for high-frequency trading, market analysis, and maintaining competitive advantage (QuestDB, 2019).

optimal conditions. By having cash reserves, the model does not lose the option to deploy capital at the time of positive conditions that generate high conviction signals.

- **Execution Friction and Slippage Sensitivity: T**o understand how slippage interferes with and affects execution friction in our model for liquidity stress testing, it is crucial to highlight how sensitive the model's performance is to market impact and execution costs. In this context, slippage tends to increase in case of a flash crash in the market since the execution cost of high-frequency trading in this case is around 12 basis points. This means that slippage will result in approximately a 24% drop in the net adjusted Sharpe Ratio from 1.74 to 1.34. This relationship shows that the success of the proposed model will be contingent on sound execution architecture and optimal order routing when applied in high frequency environments.
- **Market Bias:** The evaluation period during part of 2023 consisted of a complicated macroeconomic situation. While it was broadly characterized by a prosperous bull market for technology companies backed by the remarkable rise of stock prices of companies such as Microsoft and NVIDIA, the process was not uniform. The market was undergoing bearish cycles at the same time as significant volatility due to aggressive interest rate hikes and concerns about inflation was witnessed. The framework managed to overcome the negative inflationary periods given its primary focus of is risk management rather than mere profit maximization, the fact that it managed to maintain capital during periods of downturn validates its defense function. However, it remains to be seen how the implementation will carry on as it should be able to adapt to different macroeconomic regimes.

Even if these operational limits reveal the current limits of the Minimum Viable Product, they should not be seen as opposing its basic accuracy, as they actually only show what infrastructure and execution improvements need to be made to implement this defensive multimodal architecture completely.

Chapter 6. CONCLUSIONS

6.1 Conclusions of the work

The general objective of this project was to create a prescriptive analytics model that combines Natural Language Processing (NLP) of sentiment analytics with macroeconomic and technical indicators for optimized investment decision-making and protecting capital during volatility. The results presented in this paper demonstrate that the project has achieved its goal.

The FinBERT language model produced a continuous sentiment score for a grand corpus of financial news. This is a significant contribution to the solution of the “modality gap” problem which often happens when using Natural Language Processing (NLP) techniques that are trained with general purpose information. With the daily sentiment score and quantitative metrics (VIX for volatility and detect fear in the market, volume for liquidity identification, and RSI for clarification of oversold or overbought conditions of the stock), a multi-modal dataset is created that allows XGboost classification engine to operate effectively.

The adoption of the Triple Barrier Labeling (TBL) allows the model to simulate real market conditions, where stock prices do not move in a straight line. For example, a trade might experience a sharp drop on day two, but then recover to hit the profit target by day four. Traditional static models fail to capture this reality as they try to predict a static price in the future, but TBL accounts for this path-dependency by setting volatility adjusted take-profit and stop-loss limits to manage intraday volatility. These labels were then used in combination with the Fractional Kelly Criterion (with a multiplier of 0.6 to limit exposure) and a constraint of using a maximum of 35% of cash available per trade to limit risk, is a conservative filter. It ensures the model only operates with high conviction trades and safely switches to cash during high risk periods.

During the evaluation period, the XGBoost strategy obtained a cumulative return of 29.19%, compared to a 25.97% return from the S&P 500 benchmark. The strategy has achieved this profitability with less risk because its maximum historical drawdown is -5.42% versus -9.97% of the benchmark. Also, it is important to consider that it has a low Beta of 0.11 and an Annualized Jensen Alpha of 14.25%, which shows that the returns are generated by the model's predictive capacity, rather than general market momentum or just following trends. This means that the strategy's results are beneficial as they are not affected by passive market

movements, confirming that a machine learning based and sentiment aware approach to financial trading is a robust solution to assess the challenges of modern financial markets.

6.2 Personal conclusions

The development of the master's in business analytics program and the project itself has been an unbelievable experience of transformation. We each had a foundation in the administrative sciences, banking and finance, and thus moving into an area with very technical aspects of deep learning, data engineering, and predictive algorithms required us to recognize that we needed to adjust the way we look at financial systems. We have moved away from traditional financial management and treasury-style positions where the industry still uses manual tracking, outdated spreadsheet models and legacy quantitative assumptions. This project has allowed us to step away from the traditional methods of financial management and has opened the door to the great benefits of having the entire process automated and using prescriptive analytics.

The learning curve during the research and development stages was steep, but it also provided great rewards. Transforming the original theoretical concepts of the finance field into constructing solid machine learning pipelines represented the major transition from our previous area of expertise. By operating within a python environment and utilizing powerful machine learning frameworks such as XGBoost and Scikit-learn and by overcoming engineering obstacles associated with out-of-core data processing, we constructed machine learning pipelines that allowed us to process data sets with more than 15 million records. The ability of a transformer style model (FinBERT) to "read" and analyze complex financial statements, while producing real alpha indicators, was an incredible step towards helping define how far technologically advanced financial systems have come to date.

This project has ultimately had a significant effect on our professional development. We now possess the technical fluency and strategic mindset necessary to function in the rapidly evolving field of data science in institutional finance. We have completed our thesis not only as professionals in the financial services industry but also as quantitative analysts prepared to develop automated and data-based applications that will shape the future of the financial services industry.

Chapter 7. FUTURE WORK

The current work works well as an MVP and has accomplished its objective of bridging the gap in modalities of quantitative analysis. However, through its success, numerous strategic options for improvements to the prescriptive framework have emerged.

The next step for future iterations should focus on using dynamic exit barriers. Analysis of execution data revealed that the predominant reason for closing of the majority of trades is through a "Time-Out" condition, therefore, by implementing a dynamic limit, based upon current volatility levels versus using a static profit target, allows greater flexibility to the closing of trades in environments of high volatility.

Another area of future research is to use real time information like financial news and prices, rather than historical data. With the introduction of Application Programming Interfaces (APIs), as those by Bloomberg and Reuters, the framework would be very advanced in processing news and market prices in real-time. Future work may explore if the framework is able to keep its predicting capacity and profitability while operating on live. Also, using the model with this real time escalation might cause a huge computational capacity problem since this is a MVP with a limited computing capacity. An upgrade of the computational infrastructure is required to start working with real time feeds to allow the model to analyze a massive quantity of financial information without experiencing any performance delays.

Future developments should focus on an evolution of the portfolio allocation logic from a static fractional boundary to a dynamic regime aware method. While the current implementation correctly scales position sizes according to the XGBoost predicted probability, the baseline 0.60 Fractional Kelly multiplier is still static at this time. Future work could evolve by automatically adjusting the multiplier based upon real time macroeconomic volatility, for instance, using a shorter multiplier during extreme spikes in the VIX, or longer multiplier values in stable bullish markets.

Chapter 8. REFERENCES

Alvim de Faria, M., & Lundquist, F. (2025). Financial sentiment analysis: Evaluated on the stock market. A comparative study between large-language models and transformer-based sentiment classifications [Bachelor's thesis, KTH Royal Institute of Technology]. DiVA Portal. https://www.diva-portal.org/smash/get/diva2:1985458/FULLTEXT01.pdf

Antweiler, W., & Frank, M. Z. (2004). Is all that talk just noise? The information content of internet stock message boards. The Journal of Finance, 59(3), 1259–1294. https://doi.org/10.1111/j.1540-6261.2004.00662.x

Araci, D. (2019). *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models.* ArXiv. https://arxiv.org/abs/1908.10063

Bacon, C. R. (2023). *Practical portfolio performance measurement and attribution* (3rd ed.). Wiley. https://doi.org/10.1002/9781119831976

Bäuerle, N., & MacKay, A. (2026). Mean-Variance Optimization in Ambiguous Financial Markets with Learning. arXiv preprint. https://arxiv.org/html/2606.11318v1

Bhuvaneshwari, D. (2025). Predictive analytics in behavioral finance: Modeling investor sentiment with NLP techniques. *International Journal of Finance, Economics, and Management Studies, 1*(1), 19–28. https://doi.org/10.64137/31080030/IJFEMS-V1I1P103

Bodie, Z., Kane, A., & Marcus, A. J. (2024). Investments (13th ed.). McGraw Hill LLC. https://info.mheducation.com/rs/128-SJW-347/images/Bodie_Preface_Investments_13e.pdf

Bollen, J., Mao, H., & Zeng, X. (2011). *Twitter mood predicts the stock market*. Journal of Computational Science, 2(1), 1–8. https://doi.org/10.1016/j.jocs.2010.12.007

Cezanne, P. (2024). *Large language model-driven sentiment analysis for exploring the influence of social media and financial news on stock market development* [Master's thesis,

Otto-Friedrich-University Bamberg]. https://www.uni-bamberg.de/fileadmin/xai/studies/theses/2024/2024_Masterthesis_Cezanne.pdf

Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Dixon, M. F., Halperin, I., & Bilokon, P. (2020). *Machine learning in finance: From theory to practice*. Springer. https://link.springer.com/book/10.1007/978-3-030-41068-1

Dong, Z., Fan, X., & Peng, Z. (2024). FNSPID: A Comprehensive Financial News Dataset in Time Series. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 4918–4927. https://doi.org/10.1145/3637528.3671629

Dong, Z., Fan, X., & Peng, Z. (2024). *FNSPID_Financial_News_Dataset* [Source code and data set]. GitHub. https://github.com/Zdong104/FNSPID_Financial_News_Dataset

Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417. https://doi.org/10.2307/2325486

Fischer, T., & Krauss, C. (2017). *Deep learning with long short-term memory networks for financial market predictions.* European Journal of Operational Research, 270(2), 654–669. https://doi.org/10.1016/j.ejor.2017.11.054

Fu, N., Kang, M., Hong, J., & Kim, S. (2024). Enhanced genetic-algorithm-driven triple barrier labeling method and machine learning approach for pair trading strategy in cryptocurrency markets. *Mathematics*, 12(5), Article 780. https://doi.org/10.3390/math12050780

Fu, N., Kang, M., Hong, J., & Kim, S. (2026). *Explainable algorithmic trading with embedded intelligence. International Journal of Accounting & Information Management,* 34(2), 145–162. https://doi.org/10.1108/IJAIM-10-2025-0351

Goldstein, I., Spatt, C. S., & Ye, M. (2021). *Big data in finance. The Review of Financial Studies, 34*(7), 3213–3225. https://doi.org/10.1093/rfs/hhab038

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. The MIT Press. https://www.researchgate.net/publication/320703571_Ian_Goodfellow_Yoshua_Bengio_and_Aaron_Courville_Deep_learning_The_MIT_Press_2016_800_pp_ISBN_0262035618

Harris, L. (2003). Trading and exchanges: Market microstructure for practitioners. Oxford University Press. https://www.acsu.buffalo.edu/~keechung/MGF743/Readings/Trading-Exchanges-Market-Microstructure-Practitioners%20Draft%20Copy.pdf

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hilpisch, Y. (2018). *Python for Finance: Mastering Data-Driven Finance*. O'Reilly Media. https://www.sea-stat.com/wp-content/uploads/2021/05/Yves-Hilpisch-Python-for-Finance_-Mastering-Data-Driven-Finance-Book-OReilly-2018.pdf

Hull, J. C. (2022). *Options, futures, and other derivatives* (11th ed.). Pearson. http://lib.ysu.am/disciplines_bk/2b66030e0dd4c77b2bda437f6c1e5e66.pdf

IBM. (2024). *What is XGBoost?* https://www.ibm.com/think/topics/xgboost

Interactive Brokers (2024). *The Triple Barrier Method: A Python GPU-based Computation - Part I.* IBKR Campus US. https://www.interactivebrokers.com/campus/ibkr-quant-news/the-triple-barrier-method-a-python-gpu-based-computation-part-i/

Jensen, M. C. (1968). The performance of mutual funds in the period 1945–1964. *The Journal of Finance*, 23(2), 389–416. https://doi.org/10.1111/j.1540-6261.1968.tb00815.x

Kang, S. (2025). *Stock price prediction using triple barrier labeling and raw OHLCV data: Evidence from Korean markets*. arXiv. https://arxiv.org/abs/2504.02249

Kelly, J. L. (1956). A New Interpretation of Information Rate. *Bell System Technical Journal*, 35(4), 917-926. https://www.princeton.edu/~wbialek/rome/refs/kelly_56.pdf

Lauricella, T., & Schlueter, R. (2026). *3 Years of the AI Stock Market Boom in Charts* | Morningstar Europe. Morningstar. https://global.morningstar.com/en-eu/stocks/3-years-ai-stock-market-boom-charts

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* arXiv. https://arxiv.org/abs/1907.11692

López de Prado, M. (2018). *Advances in financial machine learning* [Supplemental material]. Gildan Media. https://gildan-bonus-content.s3.amazonaws.com/GIL2476_AdvancesFinancial/GIL2476_AdvancesFinancial_BonusPDF.pdf

Loughran, T., & McDonald, B. (2011). When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. *The Journal of Finance*, 66(1), 35–65. https://doi.org/10.1111/j.1540-6261.2010.01625.x

Muhammad, I., Rospocher, M., Knez, T., & Žitnik, S. (2025). Benchmarking large language models for target-based financial sentiment analysis. *In Proceedings of the Italian Conference on Computational Linguistics (CLiC-it)*. Association for Computational Linguistics. https://aclanthology.org/2025.clicit-1.74.pdf

Murphy, J. J. (1999). *Technical analysis of the financial markets. Penguin Random House.* https://www.penguinrandomhouse.com/books/330777/technical-analysis-of-the-financial-markets-by-john-j-murphy/

Narang, R. K. (2013). *Inside the Black Box: A Simple Guide to Quantitative and High-Frequency Trading.* John Wiley & Sons. https://content.e-bookshelf.de/media/reading/L-24402424-11c92b354b.pdf

Noble, J. (2025). *Long short-term memory*. Ibm.Com. https://www.ibm.com/think/topics/lstm

Omoseebi, A., Owen, J., & Loveth, Q. (2023). *Natural language processing (NLP) for sentiment analysis in financial markets.* ResearchGate. https://www.researchgate.net/publication/388848099_Natural_language_processing_NLP_for_sentiment_analysis_in_financial_markets

NVIDIA. (2022). *NVIDIA H100 Tensor Core GPU architecture* [White paper]. https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c

Out of core computation — strax 2.2.2 documentation. (2018). Readthedocs.Io. https://strax.readthedocs.io/en/latest/advanced/out_of_core.html

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., & Dean, J. (2022). *Efficiently scaling transformer inference* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2211.05102

Priya, P., Pravallika, G., & Final, M. (2024). *Machine Learning Model for Sales Forecasting by using XGBoost.* 11. https://www.jetir.org/papers/JETIR2407382.pdf

Qinyun Zen. (2019). *Financial sentiment analysis using FinBERT with application in predicting stock movement*. Arxiv.Org. https://arxiv.org/html/2306.02136v2

QuestDB. (2019). *QuestDB*. https://questdb.com/glossary/tick-data/

Randall, D. (2022). *Whispers of S&P 500 bear market grow louder as U.S. stock decline continues.* Reuters. https://www.reuters.com/business/whispers-sp-500-bear-market-grow-louder-us-stock-decline-continues-2022-05-09/

Reddy, T. (2025). *DRAWBACKS OF LSTM ALGORITHM: A CASE STUDY.* https://doi.org/10.13140/RG.2.2.32515.16166

Rijal, S., Cakranegara, P. A., Ciptaningsih, E. M. S. S., Pebriana, P. H., Andiyan, A., & Rahim, R. (2023). Integrating Information Gain methods for feature selection in distance education sentiment analysis during Covid-19. *TEM Journal*, 12(1), 285–290. https://doi.org/10.18421/TEM121-35

Ruan, L., & Jiang, H. (2025). Stock Price Prediction Using FinBERT-Enhanced Sentiment with SHAP Explainability and Differential Privacy. *Mathematics*, 13(17), 2747–2747. https://doi.org/10.3390/math13172747

Schrádi, T., & Juhász, S. (2009). *Out-of-core processing in preparation phase of data mining tasks. In Proceedings of the 10th International Symposium of Hungarian Researchers on Computational Intelligence and Informatics* (pp. 1 - 559). https://www.researchgate.net/publication/229001495_Out-of_Core_Processing_in_Preparation_Phase_of_Data_Mining_Tasks

Schrádi, T., Dudás, Á., & Juhász, S. (2011). *Optimization of out-of-core data preparation methods identifying runtime determining factors.* In Proceedings of the 2011 International

Conference on Foundations of Computer Science (FCS) [(pp. 1–7)]. CSREA Press. https://worldcomp-proceedings.com/proc/p2011/FCS4904.pdf

Sharpe, W.F. (1994) The Sharpe Ratio. *The Journal of Portfolio Management*, 21, 49-58. http://dx.doi.org/10.3905/jpm.1994.409501

Singh Chauhan, M. K. (2024). *A Tale of Three Models: How Transformers, BERT, and ULMFiT* Took NLP by Storm. Medium. https://medium.com/@mukul.mschauhan/a-tale-of-three-models-how-transformers-bert-and-ulmfit-took-nlp-by-storm-58080f1459b6

Skelton, J. (2022). *Tensor Cores Explained in Simple Terms.* Digitalocean.Com; DigitalOcean. https://www.digitalocean.com/community/tutorials/understanding-tensor-cores

Sortino, F. A., & Price, L. N. (1994). Performance Measurement in a Downside Risk Framework. *The Journal of Investing*, 3(3), 59-64. https://www.pm-research.com/content/iijinvest/3/3/59

State Street Global Advisors. (2026). SPDR S&P 500 ETF Trust (SPY) Prospectus. State Street Corporation. https://www.ssga.com/us/en/intermediary/etfs/state-street-spdr-sp-500-etf-trust-spy

Tadrzak, A. (2020). *The COVID-19 Impact on eCommerce: How to Adapt your Online Store.* Datafeedwatch.Com; DataFeedWatch. https://www.datafeedwatch.com/blog/how-coronavirus-changes-ecommerce

Tetlock, P. C. (2007). *Giving content to investor sentiment: The role of media in the stock market.* The Journal of Finance, 62(3), 1139–1168. https://doi.org/10.1111/j.1540-6261.2007.01232.x

Thorp, E. O. (2006). The Kelly criterion in blackjack, sports betting, and the stock market. In S.A. Zenios & W. T. Ziemba (Eds.), *Handbook of Asset and Liability Management: Volume 1:*

Theory and Methodology (pp. 385–428). Elsevier. https://gwern.net/doc/statistics/decision/2006-thorp.pdf

TradingView. (2026). *Profit factor. TradingView.* https://es.tradingview.com/support/solutions/43000681698/

Treveil, M., Wilson, N., Zisler, M., & The Dataiku Team. (2020). I*ntroducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media. https://itsocial.fr/wp-content/uploads/2021/04/Comment-mettre-%C3%A0-l%E2%80%99%C3%A9chelle-le-Machine-Learning-en-entreprise.pdf

Tür, A. Ş. (2024). *Financial sentiment analysis in annual reports of BIST100 companies: Comparison of dictionary-based and deep learning-based methods* [Master's thesis, Middle East Technical University]. Open METU. https://open.metu.edu.tr/bitstream/handle/11511/113400/10565277.pdf

U.S. Securities and Exchange Commission. (2023). *SEC Charges UAE-Based Brooge Energy and Former Executives with Fraud.* U.S. Securities and Exchange Commission. https://www.sec.gov/newsroom/press-releases/2023-256

Van Mook, T. (2025). *Market reactions to earnings calls: Evidence from GPT-4o derived sentiment analysis* [Master's thesis, Tilburg University]. http://arno.uvt.nl/show.cgi?fid=188999

Vaswani, A., Shazeer, N., Parmar N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762

Whaley, R. E. (2009). Understanding the VIX. *The Journal of Portfolio Management*, 35(3), 98-105. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1296743

Wilder, J. W. (1978). *New concepts in technical trading systems* [1a. ed. española]. Trend Research. https://archive.org/details/newconceptsintec00wild

Xiao, X., Wang, F., Xu, H., Wang, D., & Zhang, Y. (2025). Applying XGBoost for Time Series Prediction in Financial Market Data. https://doi.org/10.21203/rs-7017751/v1

Xing, X., Wang, Z., & Zhang, M. (2025). Optimal Kelly Portfolio under Risk Constraints. *Engineering*, 17(03), 222–240. https://doi.org/10.4236/eng.2025.173014

Yahoo! Finance. (n.d.). *Brookfield Property Partners L.P. (BPYPO) stock price, news, quote & history.* https://finance.yahoo.com/quote/BPYPO/

Yang, Y., Uy, M. C., & Huang, A. (2020). *FinBERT: A pretrained language model for financial* communications. arXiv. https://arxiv.org/abs/2006.08097

yfinance. (2026). PyPI. https://pypi.org/project/yfinance/

Chapter 9. APPENDICES

Appendix A: Out-of-Core Data Processing Architecture

```
def discover_and_analyze_news_chunked(filepath: str, top_n: int = 7, chunk_size: int = 500000, start_date:
                                      str = '2018-01-01', end_date: str = '2023-12-31') -> list:
    # 1. Map Schema dynamically (In-Memory footprint: ~1KB)
    sample_df = pd.read_csv(filepath, nrows=1)
    date_col_candidates = ['date', 'time', 'timestamp', 'publish_date']
    ticker_col_candidates = ['ticker', 'symbol', 'stock_symbol', 'stock']

    date_col = next((c for c in sample_df.columns if c.lower() in date_col_candidates), None)
    ticker_col = next((c for c in sample_df.columns if c.lower() in ticker_col_candidates), None)

    if not date_col or not ticker_col:
        raise ValueError(f"Could not map required columns. Found: {list(sample_df.columns)}")

    # 2. Initialize an empty Series to act as our global accumulator
    total_counts = pd.Series(dtype='int64')
```

```
    # 4. Stream and Process
    for chunk in chunk_iterator:

        # Standardize column names for the current chunk
        chunk.rename(columns={date_col: 'Date', ticker_col: 'Ticker'}, inplace=True)

        # Clean data
        chunk['Date'] = chunk['Date'].str.strip()
        chunk['Ticker'] = chunk['Ticker'].str.strip().str.upper()
        chunk.dropna(subset=['Date', 'Ticker'], inplace=True)

        # Temporal filtering
        chunk['Date'] = chunk['Date'].astype(str).str[:10]
        mask = (chunk['Date'] >= start_date) & (chunk['Date'] <= end_date)
        filtered_chunk = chunk.loc[mask]

        # Calculate frequencies for THIS chunk only
        chunk_counts = filtered_chunk['Ticker'].value_counts()

        # Add the chunk's counts to the global accumulator
        total_counts = total_counts.add(chunk_counts, fill_value=0)


    # 5. Final Sorting and Extraction
    total_counts = total_counts.sort_values(ascending=False).astype(int)

    top_tickers_df = total_counts.head(top_n).reset_index()
    top_tickers_df.columns = ['Ticker', 'Article_Count']
    top_tickers_list = top_tickers_df['Ticker'].tolist()

    return top_tickers_list
```

Appendix B: Quantitative Feature Engineering

```python
def calculate_native_rsi(prices: pd.Series, window: int = 14) -> pd.Series:
    delta = prices.diff()
    gain = delta.where(delta > 0, 0.0)
    loss = -delta.where(delta < 0, 0.0)
    avg_gain = gain.ewm(alpha=1/window, adjust=False).mean()
    avg_loss = loss.ewm(alpha=1/window, adjust=False).mean()
    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))

def fetch_and_engineer_features(tickers: list, start_date: str, end_date: str) -> pd.DataFrame:
    print(f"Downloading complete OHLC for {len(tickers)} assets...")
    # Download raw data
    df_raw = yf.download(tickers, start=start_date, end=end_date, progress=False)

    # Convert to long format (Tidy) ensuring Open, High, Low and Close are kept
    df = df_raw.stack(level=1, future_stack=True).rename_axis(['Date', 'Ticker']).reset_index()
    df.columns = df.columns.astype(str)

    # --- CRITICAL FIX APPLIED HERE ---
    # Strip timezone data to prevent fatal crashes during the Lead-Lag merge later
    df['Date'] = pd.to_datetime(df['Date']).dt.tz_localize(None).dt.normalize()
    # ---------------------------------

df = df.sort_values(by=['Ticker', 'Date']).reset_index(drop=True)

print("Feature Engineering: Gaps, RSI, and Volatility...")

# 1. Daily return (based on Close)
df['Returns'] = df.groupby('Ticker')['Close'].pct_change()

# 2. Overnight Gap (Today's Open vs Yesterday's Close)
df['Prev_Close'] = df.groupby('Ticker')['Close'].shift(1)
df['Overnight_Gap'] = (df['Open'] - df['Prev_Close']) / df['Prev_Close']

# 3. 20-day Volatility (Annualized) - Key for dynamic barriers
df['Vol_20d'] = df.groupby('Ticker')['Returns'].transform(
    lambda x: x.rolling(window=20).std() * np.sqrt(252)
)

# 4. Native RSI
df['RSI_14'] = df.groupby('Ticker')['Close'].transform(
    lambda x: calculate_native_rsi(x, window=14)
)

# Initial cleanup of NaNs due to rolling windows calculation
df.dropna(subset=['Overnight_Gap', 'Vol_20d', 'RSI_14'], inplace=True)

    return df

# ==========================================
# Market Block Execution
# ==========================================
if __name__ == "__main__":
    TARGET_TICKERS = ['AAPL', 'MSFT', 'NVDA', 'TSLA', 'GOOG', 'INTC', 'AMZN']

    df_market_data = fetch_and_engineer_features(
        tickers=TARGET_TICKERS,
        start_date='2018-01-01',
        end_date='2023-12-31'
    )

    print("\n✅ Market Data structure ready for Triple Barrier.")
    # Show critical columns for TBL
    cols_visualizacion = ['Date', 'Ticker', 'Open', 'High', 'Low', 'Close', 'Overnight_Gap', 'Vol_20d', 'RSI_14']
    print(df_market_data[cols_visualizacion].head())
```

Appendix C: NLP Deep Learning Inference Engine (FinBERT)

```
import pandas as pd
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from tqdm import tqdm

# 1. Dataset structure for high-speed processing
class FinancialNewsDataset(Dataset):
    def __init__(self, texts, tokenizer, max_length=512):
        self.texts = texts
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            str(self.texts[idx]),
            add_special_tokens=True,
            max_length=self.max_length,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten()
        }

# 2. Main Inference Engine
def run_finbert_inference(input_file: str, output_file: str, batch_size: int = 64):
    print(f"Reading {input_file}...")
    df = pd.read_csv(input_file)

    # Identify article text column
    text_col = next((c for c in df.columns if c.lower() in ['article', 'text', 'headline']), None)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Hardware Accelerator: {device.type.upper()}")
```

```
print("Loading FinBERT Model...")
tokenizer = AutoTokenizer.from_pretrained("ProsusAI/finbert")
model = AutoModelForSequenceClassification.from_pretrained("ProsusAI/finbert").to(device)
model.eval()

dataset = FinancialNewsDataset(df[text_col].tolist(), tokenizer)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=False)

scores = []
print(f"Analyzing sentiment for {len(df):,} articles...")

with torch.no_grad():
    for batch in tqdm(dataloader, desc="Inference Progress"):
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)

        outputs = model(ids, attention_mask=mask)
        probs = F.softmax(outputs.logits, dim=1)

        # Continuous Signal: Probability(Positive) - Probability(Negative)
        # Result is between -1 (Pure Bearish) and +1 (Pure Bullish)
        sentiment_signal = (probs[:, 0] - probs[:, 1]).cpu().numpy()
        scores.extend(sentiment_signal)


    df['Sentiment_Score'] = scores
    # Drop raw text to keep the final dataset lightweight
    df_final = df.drop(columns=[text_col])
    df_final.to_csv(output_file, index=False)
    print(f"✅ Analysis Complete. Scored data saved to: {output_file}")
    return df_final

# Execute
if __name__ == "__main__":
    df_sentiment = run_finbert_inference(
        input_file='Target_News_Cluster.csv',
        output_file='Sentiment_Scored_News.csv'
    )
```

Appendix D: Temporal Alignment and Lead-Lag Synchronization

```
import pandas as pd

def sync_lead_lag_data(sentiment_file, market_df):
    print("Loading sentiment scores...")
    df_s = pd.read_csv(sentiment_file)

    # 1. Columns Standarization
    ticker_col = next((c for c in df_s.columns if c.lower() in ['stock_symbol', 'ticker', 'symbol']), None)
    date_col = next((c for c in df_s.columns if c.lower() in ['date', 'time', 'timestamp', 'publish_date']), None)

    if not ticker_col or not date_col:
        print(f"❌ Error: Could not find Ticker or Date in news file. Columns found: {list(df_s.columns)}")
        return None

    df_s = df_s.rename(columns={ticker_col: 'Ticker', date_col: 'Date'})
    print(f"Standardized columns: '{ticker_col}' -> 'Ticker', '{date_col}' -> 'Date'")

    # 2. Formating dates and Time zone
    # Convertimos a datetime
    df_s['Date'] = pd.to_datetime(df_s['Date'])
    market_df['Date'] = pd.to_datetime(market_df['Date'])
```

```
# Deleting timezone naive
df_s['Date'] = df_s['Date'].dt.tz_localize(None)
market_df['Date'] = market_df['Date'].dt.tz_localize(None)

df_s['Date'] = df_s['Date'].dt.normalize()
market_df['Date'] = market_df['Date'].dt.normalize()

# 3. Calculating daily mean sentiment per ticker
print("Calculating daily mean sentiment per ticker...")
df_daily_sent = df_s.groupby(['Ticker', 'Date'])['Sentiment_Score'].mean().reset_index()

# 4. El Desfase (LEAD-LAG): Yesterday´s Sentiment (T) effect today (T+1)
df_daily_sent['Sync_Date'] = df_daily_sent['Date'] + pd.Timedelta(days=1)
```

```
 # 5. Merge Final
 print("Merging Market Data with Lagged Sentiment Signal...")
 df_final = pd.merge(
     market_df,
     df_daily_sent[['Ticker', 'Sync_Date', 'Sentiment_Score']],
     left_on=['Date', 'Ticker'],
     right_on=['Sync_Date', 'Ticker'],
     how='inner'
 )

 df_final.rename(columns={'Sentiment_Score': 'Lagged_Sentiment_Tminus1'}, inplace=True)
 df_final.drop(columns=['Sync_Date'], inplace=True)

 # 7. Encoding Tickers for XGBoost Compatibility..."
 print("Encoding Tickers for XGBoost Compatibility...")
 df_final['Ticker_Code'] = df_final['Ticker'].astype('category').cat.codes

 print(f"✅ Lead-Lag Synchronization Complete. Final Rows: {len(df_final):,}")
 return df_final
```

```
# Execution
# ========================================
if __name__ == "__main__":
    df_master_synced = sync_lead_lag_data('Sentiment_Scored_News.csv', df_market_data)

    if df_master_synced is not None:
        df_master_synced.to_csv('Master_Dataset_Lagged.csv', index=False)
        print("\nPreview of the Final Aligned Dataset:")
        print(df_master_synced[['Date', 'Ticker', 'Ticker_Code', 'Lagged_Sentiment_Tminus1', 'Overnight_Gap', 'Vol_20d']].head())
```

Appendix E: Triple Barrier Labeling

```
import pandas as pd
import numpy as np
import warnings

warnings.filterwarnings('ignore')

def apply_triple_barrier_labels(df_entries, df_full_market, pt_multiplier=2.0, sl_multiplier=1.0, days_horizon=5):
    """
    Implements the Triple Barrier Method for prescriptive labeling.
    Uses an asymmetrical 2:1 Reward-to-Risk ratio dynamically scaled by volatility.
    """
    print(f"Applying Triple Barrier Labeling on {len(df_entries):,} observations...")

    labels = []

    # Pre-sort and set index for O(1) lookups instead of O(N) filtering
    df_full_market = df_full_market.sort_values(['Ticker', 'Date'])

    for i, row in df_entries.iterrows():
        ticker = row['Ticker']
        entry_date = row['Date']
        entry_price = row['Open']
        vol_annual = row['Vol_20d']
```

```
    # Scale annualized volatility to the daily holding period
    daily_vol = vol_annual / np.sqrt(252)

    # 1. Define Dynamic Thresholds (Horizontal Barriers)
    upper_barrier = entry_price * (1 + (pt_multiplier * daily_vol))
    lower_barrier = entry_price * (1 - (sl_multiplier * daily_vol))

    # 2. Extract Future Window (Vertical Barrier)
    # Using boolean masking to look exactly 'days_horizon' ahead
    mask = (df_full_market['Ticker'] == ticker) & (df_full_market['Date'] > entry_date)
    future_data = df_full_market[mask].head(days_horizon)

    if len(future_data) < days_horizon:
        # If we don't have a full 5-day window (e.g., end of dataset), drop it
        labels.append(np.nan)
        continue

    label = 0  # Default: Vertical Barrier hit (Time out -> Hold/Close)
```

```
        # 3. Path Dependency Evaluation
        for _, fut_row in future_data.iterrows():
            hit_upper = fut_row['High'] >= upper_barrier
            hit_lower = fut_row['Low'] <= lower_barrier

            # Intraday Ambiguity Check: If both hit on the same day, be conservative
            if hit_upper and hit_lower:
                label = -1 # Assume Stop-Loss hit first to enforce strict risk control
                break
            elif hit_upper:
                label = 1  # Take Profit
                break
            elif hit_lower:
                label = -1 # Stop Loss
                break

        labels.append(label)

        if (i + 1) % 1000 == 0:
            print(f"   Processed {i + 1:,} rows...")

    df_entries['Target_Label'] = labels

    # Clean up unlabelable data at the end of the time series
    initial_len = len(df_entries)
    df_entries.dropna(subset=['Target_Label'], inplace=True)
    df_entries['Target_Label'] = df_entries['Target_Label'].astype(int)

    print(f"✅ Labeling Complete. Dropped {initial_len - len(df_entries)} edge-case rows.")
    return df_entries

# ==========================================
# Execution
# ==========================================
if __name__ == "__main__":
    # Parameters designed for a Prescriptive "Sniper" approach
    df_final_labeled = apply_triple_barrier_labels(
        df_entries=df_master_synced,
        df_full_market=df_market_data,
        pt_multiplier=2.0, # Asymmetrical Reward
        sl_multiplier=1.0, # Tight Risk Control
        days_horizon=5     # Liquidity Rotation limit
    )
```

```
print("\n📊 Final Target Label Distribution:")
print("-" * 35)
distribution = df_final_labeled['Target_Label'].value_counts(normalize=True) * 100
print(f"Buy (1):   {distribution.get(1, 0):.2f}%")
print(f"Sell (-1): {distribution.get(-1, 0):.2f}%")
print(f"Hold (0):  {distribution.get(0, 0):.2f}%")
print("-" * 35)

df_final_labeled.to_csv('Final_Training_Dataset.csv', index=False)
print("💾 Saved as 'Final_Training_Dataset.csv'")
```

Appendix F: XGBoost Engine

```
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from xgboost import XGBClassifier
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import classification_report, accuracy_score
import warnings

warnings.filterwarnings('ignore')

# ==========================================
# 1. XGBOOST TRAINING FUNCTION
# ==========================================
def train_xgboost_sniper(df):
    print("Step 1: Preparing Data for XGBoost...")

    y_raw = df['Target_Label']
    y_mapped = y_raw.map({-1: 0, 0: 1, 1: 2})

    features = ['Lagged_Sentiment_Tminus1', 'RSI_14', 'Vol_20d', 'VIX_Lag1', 'Ticker_Code']
    X = df[features]


    # --- CHRONOLOGICAL SPLIT ---
    unique_dates = df['Date'].sort_values().unique()
    split_date = unique_dates[int(len(unique_dates) * 0.8)]

    train_mask = df['Date'] < split_date
    test_mask = df['Date'] >= split_date

    X_train, X_test = X[train_mask], X[test_mask]
    y_train, y_test = y_mapped[train_mask], y_mapped[test_mask]

    print(f"Training on {len(X_train)} rows (Past), Testing on {len(X_test)} rows (Future)...")

    weights = compute_sample_weight(class_weight='balanced', y=y_train)
```

```
model = XGBClassifier(
    n_estimators=150,
    max_depth=5,
    learning_rate=0.03,
    objective='multi:softprob',
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

print("Step 2: Training the model...")
model.fit(X_train, y_train, sample_weight=weights)
```

```
print("Step 3: Predicting probabilities on the test set...")
probs = model.predict_proba(X_test)

y_pred_sniper = []
for p in probs:
    if p[2] > 0.42:       # High conviction Buy
        y_pred_sniper.append(2)
    elif p[0] > 0.45:     # High conviction Sell
        y_pred_sniper.append(0)
    else:                 # Low conviction -> Hold
        y_pred_sniper.append(1)

print("\n📊 Classification Report (Test Set):")
print("-" * 45)
print(classification_report(y_test, y_pred_sniper, target_names=['Sell (-1)', 'Hold (0)', 'Buy (1)']))

df_test_results = df[test_mask].copy()
df_test_results['Prediction'] = y_pred_sniper
df_test_results['Target_Label_Mapped'] = y_test

return model, df_test_results
```